

- 一、STL 概述
 - 1.1 STL 基本概念
 - 1.2 STL六大组件
 - 1.2.1 算法的分类
 - 1.2.2 迭代器
 - 1.3 vector
 - 1.3.1 for_each 迭代器
 - 1.3.2 存放自定义数据
 - 1.3.4 容器嵌套容器
- 二、常用容器
 - 2.1. string
 - 2.2 string容器常用操作
 - 2.2.1 `string`构造函数
 - 2.2.2 `string`基本赋值操作
 - 2.2.2.1 字符串的存取
 - 2.2.3 字符串拼接操作
 - 2.2.4 字符串查找和替换
 - 2.2.5 字符串比较
 - 2.2.6 字符串提取
 - 2.2.7 字符串的插入删除
 - 2.2.8 `string`和c风格的字符串转换
 - 2.3 vector容器
 - 2.3.1 `vector`的容量和大小
 - 2.3.2 `vector`的构造函数
 - 2.3.3 `vector`的赋值语句
 - 2.3.4 `vector`大小操作
 - 2.3.5 使用`swap`收缩容器容量
 - 2.3.6 `reserve`预留空间
 - 2.3.7 `vector`数据的存取
 - 2.3.8 `vector`的插入和删除
 - 2.4 deque容器
 - 2.4.1 deque容器基本概念
 - 2.4.2 deque容器实现原理
 - 2.4.3 deque常用API
 - 2.4.3.1 deque的构造和赋值
 - 2.4.3.2 deque容器的大小操作、双端插入删除操作、元素访问操作
 - 2.4.3.3 deque容器的插入和删除
 - 2.4.3.4 案例
 - 2.4.3.5 随机数
 - 2.5 stack容器
 - 2.5.1 stack容器基本概念
 - 2.5.2 stack没有迭代器
 - 2.5.3 stack常用API
 - 2.6 queue容器
 - 2.6.1 queue没有迭代器
 - 2.6.2 queue常用API

- 2.7 list链表容器
 - 2.7.1 链表的常用API
 - 2.7.2 list容器存放自定义数据
 - 2.7.3 list容器自定义数据排序，必须实现排序规则
 - 2.7.4 vector对自定义数据排序指定排序规则
 - 2.7.5 仿函数指定排序规则
 - 2.7.6 仿函数回顾
- 2.8 lambda表达式
- 2.9 set容器
 - 2.9.1 set容器常用API
 - 2.9.2 set查找操作
 - 2.9.3 set上、下限查找
 - 2.9.4 set equal_ubound查找
 - 2.9.5 对组 pair
 - 2.9.6 set更改容器的默认排序规则(推荐使用仿函数)
 - 2.9.7 set容器自定义数据必须指定排序规则,重载<运算符
 - 2.9.7 set容器自定义数据必须指定排序规则,指定排序规则
 - 2.9.8 multiset容器
- 2.10 map容器
 - 2.10.1 map容器的常用API
 - 2.10.2 map和vector容器配合使用
 - 2.10.2.1 LOL职业联赛：有4个战队，随机抽签出场，请打印出场顺序
 - 2.10.2.2 5名员工加入3个部门
- STL容器使用时机
- 三、算法
 - 3.1 函数对象
 - 3.2 谓词
 - 3.2.1 一元谓词
 - 3.2.2 二元谓词
 - 3.3 内建函数对象
 - 3.4 函数适配器
 - 3.4.1 绑定适配器
 - 3.4.2 取反适配器
 - 3.4.3 成员函数适配器
 - 3.4.4 函数指针适配器
 - 3.5 算法概述
 - 3.6 常用遍历算法
 - 3.6.1 for_each遍历算法
 - 3.6.2 transform算法
 - 3.7 常用查找算法
 - 3.7.1 find算法
 - 3.7.2 find_if算法
 - 3.7.3 adjacent_find算法
 - 3.7.4 binary_search算法
 - 3.7.5 count算法
 - 3.7.6 count_if算法

- 3.8 常用排序算法
 - 3.8.1 merge算法
 - 3.8.2 sort算法
 - 3.8.3 random_shuffle算法
 - 3.8.4 reverse算法
- 3.9 常用拷贝和替换算法
 - 3.9.1 copy算法
 - 3.9.2 replace算法
 - 3.9.3 replace_if算法
 - 3.9.4 swap算法
- 3.10 常用算术生成算法
 - 3.10.1 accumulate算法
 - 3.10.2 fill算法
- 3.11 常用集合算法
 - 3.11.1 set_intersection算法
 - 3.11.2 set_union算法
 - 3.11.3 set_difference算法
- 3.12 STL综合案例

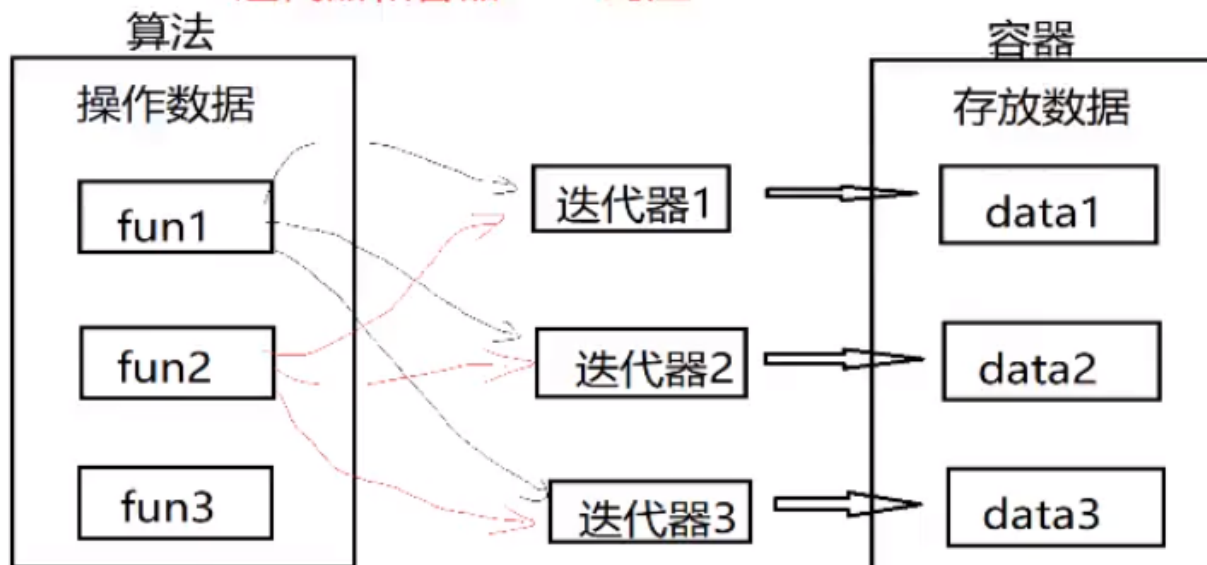
一、STL 概述

1.1 STL 基本概念

STL(Standard Template Library · 标准模板库)

STL广义上三大组件：容器(container)、算法(algorithm)、迭代器(iterator)

算法操作数据 容器存储数据 迭代器是算法操作容器的桥梁
迭代器和容器 一一对应



1.2 STL六大组件

容器(container)：存放数据，vector、list、deque、set、map等 算法(algorithm)：操作数据，sort、find、copy、for_each 迭代器(iterator)：容器和算法的桥梁，将operator*、operator->、operator++、operator--等指针相关操作予以重载的class template 仿函数(function)：为算法提供更多的策略；行为类似函数，可作为算法的某种策略。从实现角度来看，仿函数是一种重载了operator()的class或者class template. 适配器(adapter)：为算法提供更多的参数接口；用来修饰容器或者仿函数或迭代器接口的东西。空间适配器：管理容器和算法的空间；负责空间的配置与管理。从实现角度看，配置器是一个实现了动态空间配置、空间管理、空间释放的class template.

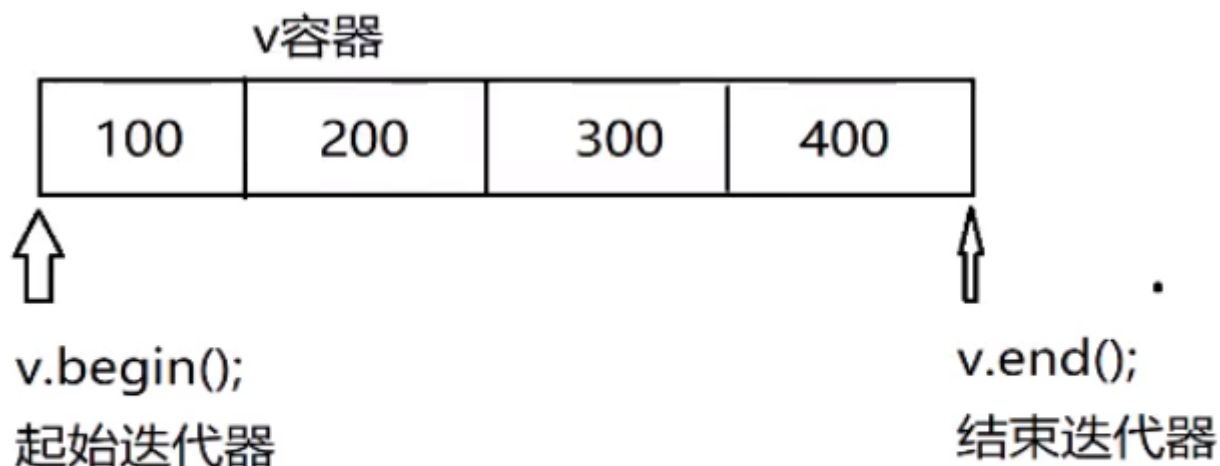
1.2.1 算法的分类

质变算法：运算过程中会更改区间内的元素的内容；拷贝、替换、删除等等。非质变算法：运算过程中不会更改容器内的数据；查找、计数、遍历、寻找极值等

1.2.2 迭代器

输入迭代器：提供对数据的只读访问；只读，支持++、==、!= 输出迭代器：提供对数据的只写访问；只写，支持++ 前向迭代器：提供读写操作，并能向前推进迭代器；读写，支持++、==、!= 双向迭代器：提供读写操作，并能向前和向后操作；读写，支持++、-- 随机访问迭代器：提供读写操作，并能以跳跃的方式访问容器的任意数据，是功能最强的迭代器；读写，支持++、[n]、-n、<、<=、>、>=

1.3 vector



起始迭代器：指向第0个元素的位置

结束迭代器：尾元素的下一个元素的位置

```
#include <iostream>
#include <vector>
#include <algorithm>

void myPrintInt(int val);

void test01()
{
    // 单端动态数组vector类模板
    std::vector<int> v; // v就是一个具体的vector容器
```

```
// push_back 尾部插入
v.push_back(100);
v.push_back(200);
v.push_back(300);
v.push_back(400);

// 访问数据
// 定义一个迭代器存储v的起始迭代器
std::vector<int>::iterator it = v.begin();
// 定义一个迭代器存储v的结束迭代器
std::vector<int>::iterator endIt = v.end();

// for循环遍历
for (; it != endIt; it++) {
    // 对迭代器取* 代表的是容器的元素
    std::cout << *it << " ";
}
std::cout << std::endl;

// for循环遍历2 (推荐)
for (std::vector<int>::iterator it = v.begin(); it != v.end(); it++) {
    std::cout << *it << " ";
}
std::cout << std::endl;

// STL提供的算法来遍历容器
// 包含算法头文件 algorithm
// for_each 从容器的起始 ---> 结束，逐个元素去取出
std::for_each(v.begin(), v.end(), myPrintInt);
std::cout << std::endl;
}

void myPrintInt(int val)
{
    std::cout << val << " ";
}

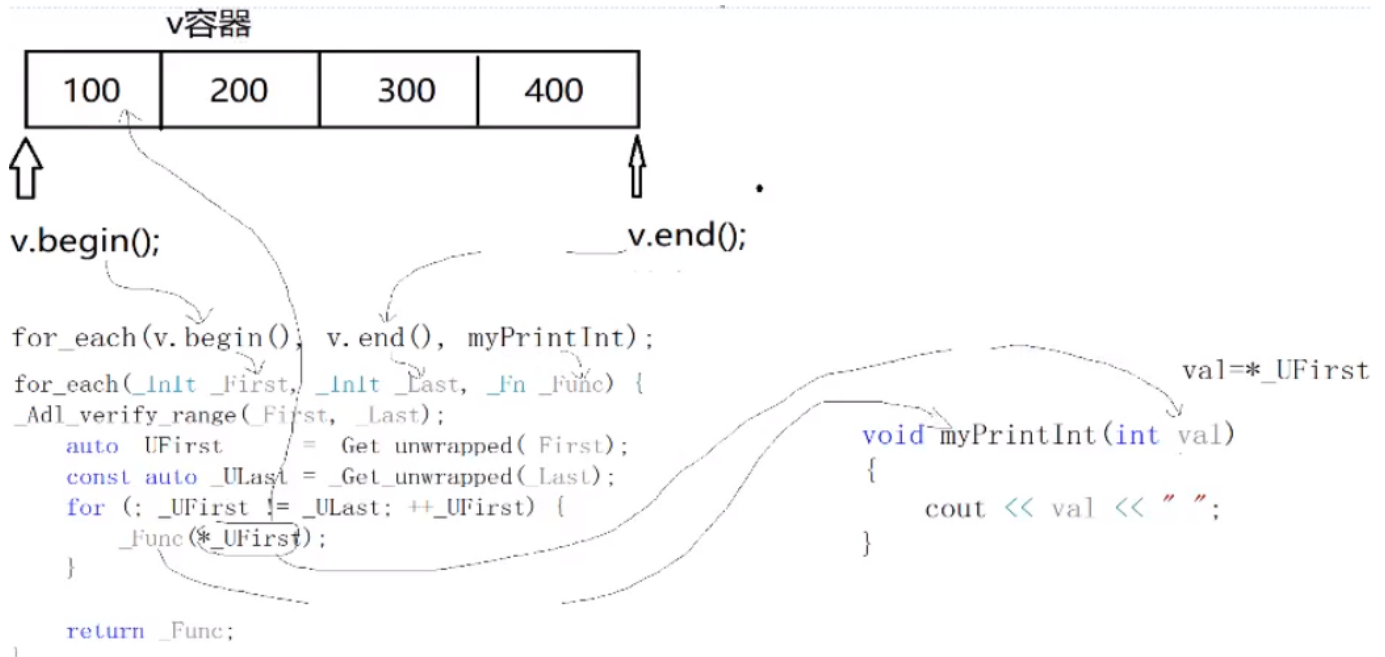
int main(int argc, char **argv)
{
    test01();

    return 0;
}
```

运行结果：

```
100 200 300 400
100 200 300 400
100 200 300 400
```

1.3.1 for_each 迭代器



1.3.2 存放自定义数据

```

class Person
{
public:
    std::string name;
    int age;
    Person(std::string name, int age) {
        this->age = age;
        this->name = name;
    }
};

void myPrintPerson(Person &ob)
{
    std::cout << "name = " << ob.name << ", age = " << ob.age << std::endl;
}

void test02()
{
    Person ob1("德玛", 18);
    Person ob2("小炮", 28);
    Person ob3("小法", 38);
    Person ob4("小李", 24);

    // 定义一个vector容器存放
    std::vector<Person> v;
    v.push_back(ob1);
    v.push_back(ob2);
    v.push_back(ob3);
    v.push_back(ob4);

    for_each(v.begin(), v.end(), myPrintPerson);
}

```

1.3.4 容器嵌套容器

```
void test03()
{
    std::vector<int> v1;
    std::vector<int> v2;
    std::vector<int> v3;

    v1.push_back(10);
    v1.push_back(20);
    v1.push_back(30);
    v1.push_back(40);

    v2.push_back(100);
    v2.push_back(200);
    v2.push_back(300);
    v2.push_back(400);

    v3.push_back(1000);
    v3.push_back(2000);
    v3.push_back(3000);
    v3.push_back(4000);

    // 需求：再定义一个vector容器，存放v1,v2,v3
    std::vector<std::vector<int>> v;
    v.push_back(v1);
    v.push_back(v2);
    v.push_back(v3);

    // for 循环遍历
    for (std::vector<std::vector<int>>::iterator it = v.begin(); it != v.end();
it++) {
        for (std::vector<int>::iterator mit = (*it).begin(); mit != (*it).end();
mit++) {
            std::cout << *mit << "\t";
        }
        std::cout << std::endl;
    }
}
```

二、常用容器

2.1. string

C风格字符串（以空字符结尾的字符数组）太过复杂难于掌握，不适合大程序的开发，所以C++标准库定义了一种string类，定在头文件。

string和C风格字符串对比：

1. `char*`是一个指针，`string`是一个类
2. `string`封装了`char*`，管理这个字符串，是一个`char`型的容器。
3. `string`封装了很多实用的成员方法：查找`find`、拷贝`copy`、删除`delete`、替换`replace`、插入`insert`
4. 不用考虑内存释放和越界
5. `string`管理`char*`所分配的内存，每一次`string`的赋值，取值都由`string`类负责维护，不用担心复制越界和取值越界等。

2.2 string容器常用操作

2.2.1 string构造函数

```
string(); // 创建一个空的字符串，例如：string str;
string(const string &str); // 使用一个string对象初始化另一个string对象
string(const char* s); // 使用字符串s初始化
string(int n, char c); // 使用n个字符串c初始化v
```

2.2.2 string基本赋值操作

```
string& operator=(const char *s); // char*类型字符串赋值给当前的字符串
string& operator=(const string &s); // 把字符串s赋给当前的字符串
string& operator=(char c); // 字符赋值给当前的字符串
string& assign(const char *s); // 把字符串s赋给当前的字符串
string& assign(const char *s, int n); // 把字符串s的前n个字符赋给当前的字符串
string& assign(const string &s); // 把字符串s赋给当前字符串
string& assign(int n, char c); // 用n个字符c赋给当前字符串
string& assign(const string &s, int start, int n); // 将s从start开始n个
```

```
int main(int argc, char **argv)
{
    // string(const char* s); // 使用字符串s初始化
    string str1("hello string");
    std::cout << str1 << std::endl;

    // string(int n, char c); // 使用n个字符串c初始化v
    string str2(10, 'a');
    std::cout << str2 << std::endl;

    string str3 = str2;
    std::cout << str3 << std::endl;

    string str4;
    // string& operator=(const string &s); // 把字符串s赋给当前的字符串
    str4 = str1;
    std::cout << str4 << std::endl;

    string str5;
    // string& operator=(const char *s); // char*类型字符串赋值给当前的字符串
```



```

    str5 = "hello str5";
    std::cout << str5 << std::endl;

    // string& operator=(char c); // 字符赋值给当前的字符串
    string str6;
    str6 = 'h';
    std::cout << str6 << std::endl;

    // string& assign(const char *s); // 把字符串s赋给当前的字符串
    string str7;
    str7.assign("hello str7");
    std::cout << str7 << std::endl;

    // string& assign(const char *s, int n); // 把字符串s的前n个字符赋给当前的字符串
    string str8;
    str8.assign("hello str8", 5);
    std::cout << str8 << std::endl;

    // string& assign(const string &s); // 把字符串s赋给当前字符串
    string str9;
    str9.assign(str8);
    std::cout << str9 << std::endl;

    // string& assign(int n, char c); // 用n个字符c赋给当前字符串
    string str10;
    str10.assign(10, 'h');
    std::cout << str10 << std::endl;

    // string& assign(const string &s, int start, int n); // 将s从start开始n个
    string str11;
    str11.assign(str7, 2, 5);
    std::cout << str11 << std::endl;

    return 0;
}

```

2.2.2.1 字符串的存取

```

char& operator[](int n); // 通过[]方式取字符
char& at(int n); // 通过at方法获取字符

```

```

void test02()
{
    string str1 = "hello string";
    cout << str1[1] << endl;
    cout << str1.at(1) << endl;

    str1[1] = 'E';
}

```

```

    cout << str1 << endl;
    str1.at(7) = 'T';
    cout << str1 << endl;

    // [] 和 at的区别
    try
    {
        // str1[1000] = 'G';    // 未捕获到异常
        str1.at(1000) = 'G';    // basic_string::at: __n (which is 1000) >= this-
>size() (which is 12)
    }
    catch(const std::exception& e)
    {
        std::cerr << e.what() << '\n';
    }
}

```

[] 和 at() 方法存在一些差异：如果访问越界，使用at()方法会抛出异常，而使用[]则不会抛出异常。

2.2.3 字符串拼接操作

```

string& operator+=(const string& str); // 重载+=操作符
string& operator+=(const char* str); // 重载+=操作符
string& operator+=(const char c); // 重载+=操作符
string& append(const char *s); // 把字符串s连接到当前字符串结尾
string& append(const char *s, int n); // 把字符串s的前n个字符连接到当前字符串结尾
string& append(const string &s); // 同operator+=( )
string& append(const string &s, int pos, int n); // 把字符串s中从pos开始的n个字符连接到当前字符串结尾
string& append(int n, char c); // 在当前字符串结尾添加n个字符c

```

```

void test03()
{
    // string& operator+=(const string& str); // 重载+=操作符
    string str1 = "hello";
    string str2 = " string";
    str1 += str2;    // str1 = str1 + str2;
    cout << str1 << endl;    // hello string

    // string& operator+=(const char* str); // 重载+=操作符
    string str3 = "hello";
    str3 += " string3";
    cout << str3 << endl;    // hello string3

    // string& operator+=(const char c); // 重载+=操作符
    string str4 = "hello";
    str4 += 'T';
    cout << str4 << endl;    // helloT
}

```

```

// string& append(const char *s); // 把字符串s连接到当前字符串结尾
string str5 = "hello";
str5.append(" str5");
cout << str5 << endl;    // hello str5

// string& append(const char *s, int n); // 把字符串s的前n个字符连接到当前字符串
// 结尾
string str6 = "hello";
str6.append("hahahahehe", 6);
cout << str6 << endl;    // hellohahaha

// string& append(const string &s); // 同operator+=( )
string str7 = "hello";
string str8 = " str8";
str7.append(str8);
cout << str7 << endl;    // hello str8

// string& append(const string &s, int pos, int n); // 把字符串s中从pos开始的n个
// 字符连接到当前字符串结尾
string str9 = "hello";
string str10 = " string10";
str9.append(str10, 5, 14); // hellong10
cout << str9 << endl;

// string& append(int n, char c); // 在当前字符串结尾添加n个字符c
string str11 = "hello";
str11.append(10, 'c');
cout << str11 << endl;    // helloccccccccc
}

```

2.2.4 字符串查找和替换

```

int find(const string& str, int pos = 0) const; // 查找str第一次出现位置，从pos开始
// 查找
int find(const char* s, int pos = 0) const; // 查找s第一次出现位置，从pos开始查找
int find(const char* s, int pos, int n) const; // 从pos位置查找s的前n个字符第一次位
// 置
int find(const char c, int pos = 0) const; // 查找字符c第一次出现位置
int rfind(const string& str, int pos = npos) const; // 查找s最后一次出现位置，从pos
// 开始查找
int rfind(const char* s, int pos = npos) const; // 查找s最后一次出现位置，从pos开始
// 查找
int rfind(const char* s, int pos, int n) const; // 从pos查找s的前n个字符最后一次位置
int rfind(const char c, int pos = 0) const; // 查找字符c最后一次出现位置
string& replace(int pos, int n, const string& str); // 替换从pos开始n个字符为字符串
// str
string& replace(int pos, int n, const char* s); // 替换从pos开始的n个字符为字符串s

```

```

void test04()
{
    // int find(const string& str, int pos = 0) const; // 查找str第一次出现位置，从
pos开始查找
    string str1 = "hehe:haha:xixi:haha:heihei";
    // 从str1中查找haha
    string tmp = "haha";
    cout << str1.find(tmp) << endl; // 5
    cout << str1.find(tmp, 10) << endl; // 15

    // int find(const char* s, int pos = 0) const; // 查找s第一次出现位置，从pos开始
查找
    cout << str1.find("hehe") << endl; // 0
    cout << str1.find("haha", 6) << endl; // 15

    // int find(const char* s, int pos, int n) const; // 从pos位置查找s的前n个字符第
一次位置
    cout << str1.find("xixi", 0, 2) << endl; // 10

    // int find(const char c, int pos = 0) const; // 查找字符c第一次出现位置
    cout << str1.find('x', 0) << endl; // 10

    // int rfind(const string& str, int pos = npos) const; // 查找s最后一次出现位
置，从pos开始查找
    cout << str1.rfind(tmp) << endl; // 15

    // int rfind(const char* s, int pos = npos) const; // 查找s最后一次出现位置，从
pos开始查找
    cout << str1.rfind("hehe", 0) << endl; // 0

    // int rfind(const char* s, int pos, int n) const; // 从pos查找s的前n个字符最后
一次位置
    cout << str1.rfind("hehexxx", 0, 4) << endl; // 0

    // int rfind(const char c, int pos = 0) const; // 查找字符c最后一次出现位置
    cout << str1.rfind('x') << endl; // 12

    // string& replace(int pos, int n, const string& str); // 替换从pos开始n个字符
为字符串str
    string tmp2 = "####";
    cout << str1.replace(5, 4, tmp2) << endl; // hehe:####:xixi:haha:heihei

    // string& replace(int pos, int n, const char* s); // 替换从pos开始的n个字符为字
符串s
    cout << str1.replace(15, 4, "####") << endl; // hehe:####:xixi:####:heihei

    // 需求：将字符串中的所有"sex"用***屏蔽
    string str2 = "www.sex.117114.sex.person.77.com";
    int pos = 0;
    while (1)
    {
        pos = str2.find("sex", pos);
        if (pos == string::npos) {

```

```

        break;
    } else {
        str2.replace(pos, strlen("sex"), "***");
    }
}
cout << str2 << endl;    // www.***.117114.***.person.77.com
}

```

2.2.5 字符串比较

`compare`函数在>时返回1, <时返回-1, ==时返回0。
比较区分大小写, 比较时参考字典顺序, 排越前面的越小。
大写的A比小写的a小。

```

int compare(const string &s) const; // 与字符串s比较
int compare(const char *s); // 与字符串s比较

```

```

void test05()
{
    string str1 = "hehe";
    string str2 = "haha";
    cout << str1.compare(str2) << endl; // 1

    cout << str1.compare("lala") << endl; // -1

    cout << str1.compare("hehe") << endl; // 0
}

```

C语言中字符串比较函数：`strcmp` C语言中内存比较函数：`memcmp`

2.2.6 字符串提取

```

string substr(int pos = 0, int n = npos) const; // 返回由pos开始的n个字符组成的字符串

```

```

void test06()
{
    string str1 = "hehe:haha:xixi:lala:heihei";
    cout << str1.substr(5, 4) << endl; // haha

    // 案例：将:分割的所有字符串提取出来
    int pos = 0;
    while (1) {
        int ret = str1.find(':', pos);
    }
}

```

```

        if (ret == string::npos) {
            cout << str1.substr(pos, ret - pos) << endl;
            break;
        }
        cout << str1.substr(pos, ret - pos) << endl;
        pos = ret + 1;
    }
}

```

2.2.7 字符串的插入删除

```

string& insert(int pos, const char* s); // 插入字符串
string& insert(int pos, const string& str); // 插入字符串
string& insert(int pos, int n, char c); // 在指定位置插入n个字符c
string& erase(int pos, int n = npos); // 删除从pos开始的n个字符

```

```

void test07()
{
    string str1 = "hello world";
    str1.insert(5, "hehe");
    cout << str1 << endl;    // hellohehe world

    cout << str1.erase(5, 4) << endl;    // hello world

    // 清空字符串
    str1.erase(0, str1.size());
    cout << str1 << endl;    //
}

```

2.2.8 string和c风格的字符串转换

```

void test08()
{
    string str1;
    char *str2 = "hello str";

    // 将char* 转成 string (直接完成)
    str1 = str2;
    cout << str1 << endl;    // hello str

    string str3 = "hello str3";
    // 不能直接将string转换成char *，必须借助string中的c_str方法完成。
    // char *str4 = str3; // error: cannot convert 'std::__cxx11::string' {aka
    // 'std::__cxx11::basic_string<char>'} to 'char*' in initialization
    char *str4 = const_cast<char*>(str3.c_str());
    cout << str4 << endl;    // hello str3
}

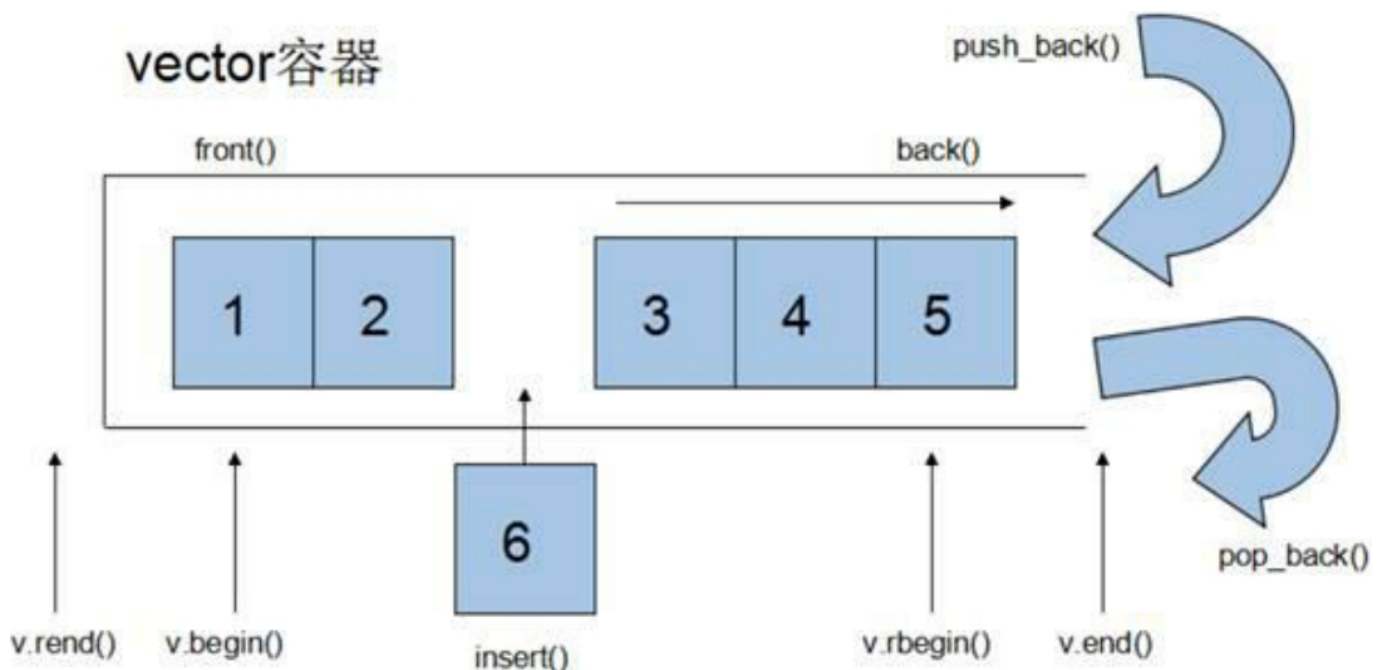
```

2.3 vector容器

vector的数据安排以及操作方式，与array非常相似，两者唯一的差别在于空间运动的灵活性。array是静态空间，一旦配置了就不能改变，要换大一点或者小一点的空间，一切琐碎都得有自己完成，首先配置一块新的空间，然后将旧空间的数据搬往新空间，再释放原来的空间。

vector是动态空间，随着元素的加入，它的内部机制会自动扩充空间以容纳新元素。因此vector的运用对于内存的合理利用与运用的灵活性有很大的帮助，我们再也不必害怕空间不足而一开始就要求一个大块头的array了。

vector的实现技术，关键在于其对大小的控制以及重新配置时的数据移动效率，一旦vector旧空间满了，如果客户每新增一个元素，vector内部只是扩充一个元素的空间，实为不智，因为所谓的扩充空间(不论大小)，是“配置新空间-数据移动-释放旧空间”的大工程，时间成本很高，应该加入某种未雨绸缪的考虑。



vector容器的迭代器：随机访问迭代器 随机访问迭代器：迭代器++可以通过编译就是随机访问迭代器

2.3.1 vector的容量和大小

capacity：空间能容纳元素的最大个数 size：空间中实际存放的元素个数

```
void test01()
{
    vector<int> v;
    int capacity = v.capacity();
    int sz = v.size();
    cout << "容器内存：" << &v[0] << "，扩充容器大小为：" << v.capacity() << endl;
    for (int i = 0; i < 100; i++) {
        v.push_back(i);
        if (v.capacity() != capacity) {
            cout << "更新了容器内存：" << &v[0] << "，扩充容器大小为：" <<
v.capacity() << endl;
            capacity = v.capacity();
        }
    }
}
```

```

    }
}

cout << "v的容量capacity: " << v.capacity() << endl;    // v的容量capacity: 128
cout << "v的大小size: " << v.size() << endl;           // v的大小size: 100
}

```

输出：

容器内存：0，扩充容器大小为：0
 更新了容器内存：0x55aa2de4c2c0，扩充容器大小为：1
 更新了容器内存：0x55aa2de4c2e0，扩充容器大小为：2
 更新了容器内存：0x55aa2de4c2c0，扩充容器大小为：4
 更新了容器内存：0x55aa2de4c300，扩充容器大小为：8
 更新了容器内存：0x55aa2de4c330，扩充容器大小为：16
 更新了容器内存：0x55aa2de4c380，扩充容器大小为：32
 更新了容器内存：0x55aa2de4c410，扩充容器大小为：64
 更新了容器内存：0x55aa2de4c520，扩充容器大小为：128
 v的容量capacity: 128
 v的大小size: 100

容量 $\geq$ 元素的个数size

为啥插入100个元素容量却是128？原因：vector的未雨绸缪机制。

只要空间满，就会另寻空间。

2.3.2 vector的构造函数

```

vector<T> v; // 采用模板实现类对象，默认构造函数
vector(v.begin(), v.end()); // 将v[begin(), end()]区间中的元素拷贝给本身
vector(n, elem); // 构造函数将n个elem拷贝给本身
vector(const vector &vec); // 拷贝构造函数

```

```

void test03()
{
    // vector(n, elem); // 构造函数将n个elem拷贝给本身
    vector<int> v1(10, 5);
    for_each(v1.begin(), v1.end(), [](int val){ cout << val << " "; });
    cout << endl;

    // vector(v.begin(), v.end()); // 将v[begin(), end()]区间中的元素拷贝给本身
    vector<int> v2(v1.begin(), v1.end() - 2);
    for_each(v2.begin(), v2.end(), [](int val){ cout << val << " "; });
    cout << endl;

    vector<int> v3(v1);
    for_each(v3.begin(), v3.end(), [](int val){ cout << val << " "; });
    cout << endl;
}

```



```

// 使用int数组赋值
int arr[] = {2, 3, 4, 1, 9};
vector<int> v4(arr, arr + sizeof(arr) / sizeof(int));
for_each(v4.begin(), v4.end(), [](int val){ cout << val << " ";}); // 2 3 4 1
9
cout << endl;
}

```

可以使用int数组赋值，int数组的变量名就是第一个元素的地址。

2.3.3 vector的赋值语句

```

assign(beg, end); // 将[beg,end)区间中的数据拷贝赋值给本身
assign(n, elem); // 将n个elem拷贝赋值给本身
vector& operator=(const vector &vec); // 重载等号操作符
swap(vec); // 将vec与自身的元素互换

```

```

void test04()
{
    vector<int> v1(5, 10);
    vector<int> v2;

    // vector& operator=(const vector &vec); // 重载等号操作符，是深拷贝。
    v2 = v1;
    printVectorInt(v2); // 10 10 10 10 10
    v2[1] = 5;
    printVectorInt(v1); // 10 10 10 10 10
    printVectorInt(v2); // 10 5 10 10 10

    // assign(n, elem); // 将n个elem拷贝赋值给本身
    vector<int> v3;
    v3.assign(5, 100);
    printVectorInt(v3);

    // assign(beg, end); // 将[beg,end)区间中的数据拷贝赋值给本身
    vector<int> v4;
    v4.assign(v3.begin(), v3.end());
    printVectorInt(v4);

    // swap(vec); // 将vec与自身的元素互换
    vector<int> v5(5, 20);
    vector<int> v6(10, 40);
    printVectorInt(v5);
    printVectorInt(v6);
    v5.swap(v6);
    printVectorInt(v5);
    printVectorInt(v6);
}

```

```
vector& operator=(const vector &vec);是深拷贝。
```

2.3.4 vector大小操作

```
size(); // 返回容器中元素的个数
empty(); // 判断容器是否为空
resize(int num); // 重新指定容器的长度为num，若容器变长，则以默认值填充新位置。
resize(int num, elem); // 重新指定容器的长度为num，若容器变长，则以elem值填充新位置。
capacity(); // 容器的容量
reserve(int len); // 容器预留len个元素长度，预留位置不初始化，元素不可访问
```

```
void test05()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);

    if (v.empty()) {
        cout << "v容器为空" << endl;
    } else {
        cout << "容器非空" << endl;
        cout << "size = " << v.size() << endl;
        cout << "capacity = " << v.capacity() << endl;
    }

    printVectorInt(v); // 10 20 30 40

    v.resize(8);
    printVectorInt(v); // 10 20 30 40 0 0 0 0
    v.resize(3);
    printVectorInt(v); // 10 20 30

    v.resize(8, 2);
    printVectorInt(v); // 10 20 30 2 2 2 2 2

    v.reserve(100); // 预留空间，但是不初始化，且不可访问
    printVectorInt(v); // 10 20 30 2 2 2 2 2
    cout << "v的容量：" << v.capacity() << endl; // 100
    cout << "v的大小：" << v.size() << endl; // 8

    v.resize(5);
    printVectorInt(v);
    cout << "v的容量：" << v.capacity() << endl; // 100, resize不会改变容器的容量
    cout << "v的大小：" << v.size() << endl; // 5

    v.resize(400, 8);
    printVectorInt(v);
```

```

    cout << "v的容量: " << v.capacity() << endl; // 400
    cout << "v的大小: " << v.size() << endl; // 400
}

```

resize作用的容器的大小，不会更改容器的容量。

2.3.5 使用swap收缩容器容量

```

void test06()
{
    vector<int> v;
    for (int i = 0; i < 1000; i++) {
        v.push_back(i);
    }
    cout << "size: " << v.size() << endl; // 1000
    cout << "capacity: " << v.capacity() << endl; // 1024

    // 使用resize将空间置为10个元素 (可以吗?)
    v.resize(10); // 不能修改容量，只能修改size
    cout << "size: " << v.size() << endl; // 10
    cout << "capacity: " << v.capacity() << endl; // 1024

    // 使用swap收缩容器的容量
    cout << vector<int>(v).size() << endl; // 10
    cout << vector<int>(v).capacity() << endl; // 10
    vector<int>(v).swap(v);
    cout << "size: " << v.size() << endl; // 10
    cout << "capacity: " << v.capacity() << endl; // 10
}

```

操作步骤：1) `v.resize(10);`先resize到10；2) `vector<int>(v).swap(v);`通过swap交换
原理：resize之后，v的size只剩下10；`vector<int>(v)`是一个匿名对象容器，将v的实际元素赋值给匿名容器，大小为10，容量也为10；`swap(v)`交换，使用匿名对象和v进行交换，交换之后，v代表匿名对象，匿名对象代表原来的v，`swap()`结束后匿名对象会被释放，相当于把原来的v对象释放了。

2.3.6 reserve预留空间

```

// reserve(int len); // 容器预留len个元素长度，预留位置不初始化，元素不可访问

```

```

void test07()
{
    vector<int> v;

    // 一次性给够空间，叫空间预留
    v.reserve(1000); // 预留空间，1000个元素
    int *p = NULL;
}

```

```

int count = 0;
for (int i = 0; i < 1000; i++) {
    v.push_back(i);
    if (p != &v[0]) {
        count++;
        p = &v[0];
    }
}
cout << "重新另寻空间次数：" << count << endl; // 11, reserve(1000)之后，只另寻了
1次
}

```

2.3.7 vector数据的存取

`at(int idx);` // 返回索引idx所指的数据，如果idx越界，抛出`out_of_range`异常。
`operator[];` // 返回索引idx所指的数据，越界时，运行直接报错
`front();` // 返回容器中的第一个数据元素
`back();` // 返回容器中的最后一个数据元素

```

void test08()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);

    printVectorInt(v); // 10
    cout << v[2] << endl; // 30
    cout << v.at(2) << endl; // 30

    // []越界不抛出异常
    // at越界抛出异常
    cout << v[100] << endl; // 0
    cout << v.at(100) << endl; // throwing an instance of 'std::out_of_range'

    cout << "front = " << v.front() << endl; // 10
    cout << "end = " << v.back() << endl; // 40
}

```

2.3.8 vector的插入和删除

`insert(const_iterator pos, int count, ele);` // 迭代器指向位置pos插入count个ele元素
`push_back(ele);` // 尾部插入元素ele
`pop_back();` // 删除最后一个元素
`erase(const_iterator start, const_iterator end);` // 删除迭代器从start到end之间的元素

```
erase(const_iterator pos); // 删除迭代器指向的元素
clear(); // 删除容器中所有元素
```

```
void test09()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);
    printVectorInt(v); // 10 20 30 40

    // insert(const_iterator pos, int count, ele); // 迭代器指向位置pos插入count个
    ele元素
    v.insert(v.begin() + 2, 3, 100);
    cout << "size = " << v.size() << endl; // 7
    cout << "capacity = " << v.capacity() << endl; // 8 · 按照动态扩充的逻辑在扩充容
    量
    printVectorInt(v); // 10 20 100 100 100 30 40

    // 尾部删除
    v.pop_back(); // 删除了40
    cout << "size = " << v.size() << endl; // 6
    cout << "capacity = " << v.capacity() << endl; // 8
    printVectorInt(v); // 10 20 100 100 100 30

    // erase(const_iterator start, const_iterator end); // 删除迭代器从start到end之
    间的元素
    // [begin, end)
    v.erase(v.begin() + 2, v.end());
    cout << "size = " << v.size() << endl; // 2
    cout << "capacity = " << v.capacity() << endl; // 8
    printVectorInt(v); // 10 20

    // erase(const_iterator pos); // 删除迭代器指向的元素
    v.erase(v.begin() + 1);
    cout << "size = " << v.size() << endl; // 1
    cout << "capacity = " << v.capacity() << endl; // 8
    printVectorInt(v); // 10

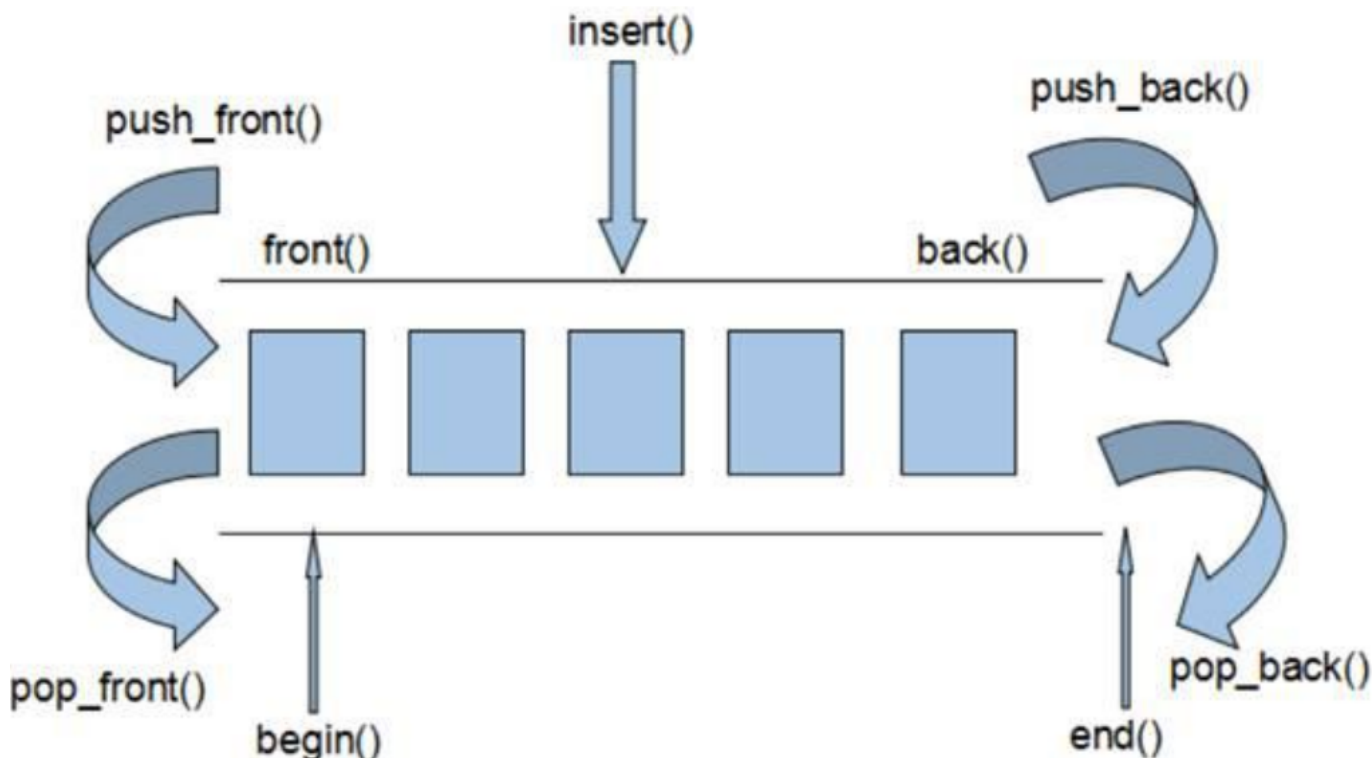
    // clear(); // 删除容器中所有元素
    v.clear();
    cout << "size = " << v.size() << endl; // 0
    cout << "capacity = " << v.capacity() << endl; // 8
}
```

2.4 deque容器

双端动态数组。

2.4.1 deque容器基本概念

vector容器是单向开口的连续内存空间，**deque**则是一种双向开口的连续性空间。所谓的双向开口，意思是在头尾两端分别做元素的插入和删除操作，当然，**vector**容器也可以在头尾两端插入元素，但是在其头部操作效率奇差，无法被接受。



deque容器和vector容器最大的差异，一在于deque允许使用常数项时间对头端进行元素的插入和删除操作。二在于deque没有容量的概念，因为它是动态的以分段连续空间组合而成，随时可以增加一段新的空间并链接起来，换句话说，像vector那样，“旧空间不足而重新配置一块更大空间，然后复制元素，再释放旧空间”这样的事在deque身上是不会发生的。也因此，deque没有必须要提供所谓的空间保留(reserve)功能。

虽然deque容器也提供了Random Access Iterator，但是它的迭代器并不是普通的指针，其复杂度和vector不是一个量级，这当然影响各个运算的层面。因此，除非有必要，应尽可能的使用vector，而不是deque。对deque进行的排序操作，为了最高效率，可将deque先完整的复制到一个vector中，对vector容器进行排序，再复制回deque中。

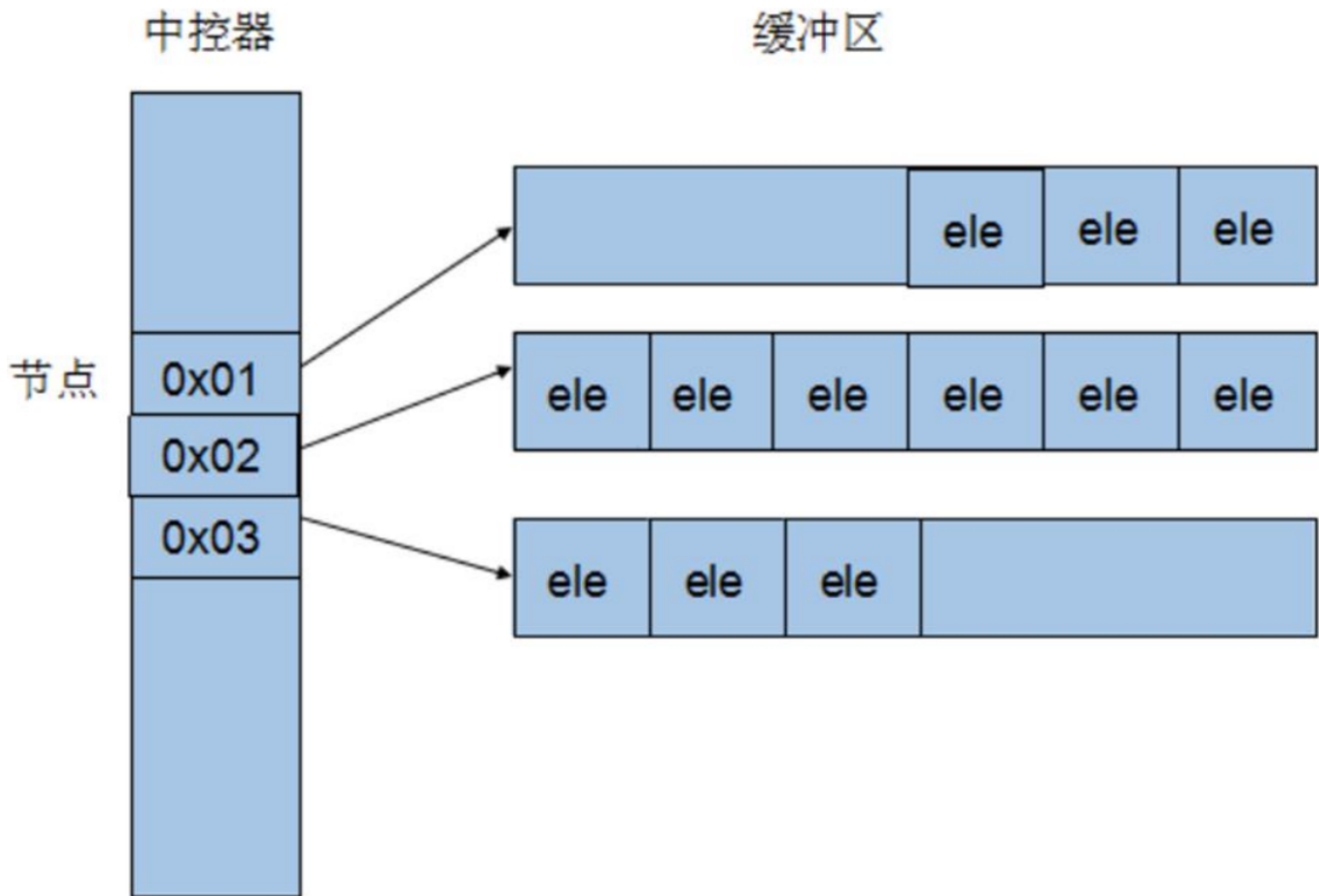
2.4.2 deque容器实现原理

deque容器是连续的空间，至少逻辑上看来如此，连续现行空间总是令我们联想到array和vector，array无法成长，vector虽可成长，却只想向尾端成长，而且其成长其实是一个假象，事实上(1)申请更大空间 (2)原数据复制新空间 (3)释放原空间 三步骤，如果不是vector每次配置新的空间时都留有余裕，其成长假象所带来的的代价是非常昂贵的。

Deque是由一段一段的定量的连续空间构成。一旦有必要在deque前端或者尾端增加新的空间，便配置一段连续定量的空间，串接在deque的头端或尾端。deque最大的工作就是维护这些分段连续的内存空间的整体性的假象，并提供随机存取的接口，避开了重新配置空间，复制，释放的轮回，代价就是复杂的迭代器架构。既然deque是分段连续内存空间，那么必须有中央控制，维持整体连续的假象，数据结构的设计及迭代器的前进后退操作颇为繁琐。

定量说明是等长的。

deque代码的实现远比vector或list都多得多。deque采取一块所谓的map(注意，不是STL的map容器)作为主控，这里所谓的map是一小块连续的内存空间，其中每一个元素(此处成为一个节点)都是一个指针，指向另一段连续性内存空间，称作缓冲区。缓冲区才是deque的存储空间主体。



2.4.3 deque常用API

2.4.3.1 deque的构造和赋值

1. deque构造函数

```
deque<T> deqT; // 默认构造形式
deque(beg, end); // 构造函数将[beg, end)区间中的元素拷贝给本身
deque(n, elem); // 构造函数将n个elem拷贝给本身
deque(const deque &deq); // 拷贝构造函数
```

2. deque赋值操作

```
assign(beg, end); // 将[beg, end)区间中的数据拷贝赋值给本身
assign(n, elem); // 将n个elem拷贝赋值给本身
deque& operator=(const deque &deq); // 重载等号操作符
swap(deq); // 将deq与本身的元素互换
```

```
void test01()
{
    deque<int> d(5, 10);
    printDequeInt(d); // 10 10 10 10 10
}
```

```

// assign(n, elem); // 将n个elem拷贝赋值给本身
deque<int> d1;
d1.assign(5, 100);
printDequeInt(d1); // 100 100 100 100 100

// deque& operator=(const deque &deq); // 重载等号操作符
deque<int> d2;
d2 = d1;
printDequeInt(d2); // 100 100 100 100 100

// swap(deq); //将deq与本身的元素互换
deque<int> d3(5, 1);
deque<int> d4(5, 2);
printDequeInt(d3); // 1 1 1 1 1
printDequeInt(d4); // 2 2 2 2 2
d3.swap(d4);
printDequeInt(d3); // 2 2 2 2 2
printDequeInt(d4); // 1 1 1 1 1

vector<int> v(10, 100);
deque<int> d5(v.begin(), v.end());
printDequeInt(d5); // 100 100 100 100 100 100 100 100 100 100
deque<int> d6;
d6.assign(v.begin(), v.end());
printDequeInt(d6); // 100 100 100 100 100 100 100 100 100 100

deque<int> d7;
d7.assign(d2.begin(), d2.end());
printDequeInt(d7); // 100 100 100 100 100=
}

```

2.4.3.2 deque容器的大小操作、双端插入删除操作、元素访问操作

3. deque大小操作

size(); // 返回容器中元素的个数
empty(); // 判断容器是否为空
resize(num); // 重新指定容器的长度为num，若容器变长，则以默认值填充新位置。
resize(num, elem); // 重新指定容器的长度为num，若容器变长，则以elem值填充

4. deque双端插入和删除操作

push_back(elem); // 在容器尾部添加一个数据
push_front(elem); // 在容器头部插入一个数据
pop_back(); // 删除容器最后一个数据
pop_front(); // 删除容器第一个数据

5. deque数据存取

at(idx); // 返回索引idx所指的数据，如果idx越界，抛出out_of_range
operator[]; // 返回索引idx所指的数据，如果idx越界，不抛出异常，直接出错
front(); // 返回第一个数据
back(); // 返回最后一个数据


```

void test02()
{
    deque<int> d;
    // 尾部插入
    d.push_back(10);
    d.push_back(20);
    d.push_back(30);

    // 头部插入
    d.push_front(40);
    d.push_front(50);
    d.push_front(60);
    printDequeInt(d); // 60 50 40 10 20 30

    // 头部删除
    d.pop_front(); // 50 40 10 20 30
    d.pop_back(); // 50 40 10 20
    printDequeInt(d); // 50 40 10 20

    if (d.empty()) {
        cout << "d容器为空" << endl;
    } else {
        cout << "d容器非空" << endl;
        cout << "size = " << d.size() << endl; // size = 4
    }

    // []方位第二个元素
    cout << "d[2] = " << d[2] << endl; // d[2] = 10
    cout << "d.at(2) = " << d.at(2) << endl; // d.at(2) = 10
    cout << "头元素 = " << d.front() << endl; // 头元素 = 50
    cout << "尾元素 = " << d.back() << endl; // 尾元素 = 20
}

```

2.4.3.3 deque容器的插入和删除

6. deque插入操作

`insert(pos, elem);` // 在pos位置插入一个elem元素的拷贝，返回新数据的位置。

`insert(pos, n, elem);` // 在pos位置插入n个elem数据，无返回值。

`insert(pos, beg, end);` // 在pos位置插入[beg,end)区间的数据，无返回值。

7. deque删除操作

`clear();` // 移除容器的所有数据

`erase(beg, end);` // 删除[beg,end)区间的数据，返回下一个数据的位置

`erase(pos);` // 删除pos位置的数据，返回下一个数据的位置

```

void test03()
{
    deque<int> d;
    d.insert(d.begin(), 5, 100);
    printDequeInt(d); // 100 100 100 100 100

    d.clear();
    cout << "size = " << d.size() << endl; // size = 0
}

```

2.4.3.4 案例

5名选手，ABCDE，10个评委分别对每一名选手打分，去除最高分，去除最低分，取平均分。

1. 创建五名选手，放到vector中；
2. 遍历vector容器，取出来每一个选手，执行for循环，可以把10个评分存到deque容器中；
3. sort算法对deque容器中分数排序，pop_back, pop_front去除最高和最低分；
4. deque容器遍历一遍，累加分数，累加分数/d.size()；
5. person.score = 平均分。

```

class Person
{
public:
    string name;
    float score;
    Person(string name, float score)
    {
        this->name = name;
        this->score = score;
    }
};

void createPerson(vector<Person> &v)
{
    // 5名选手，ABCDE
    string nameTemp = "ABCDE";
    for (int i = 0; i < 5; i++) {
        string name = "选手:";
        name += nameTemp[i];

        // 将选手的姓名 分数0 放入vector容器中
        v.push_back(Person(name, 0.0));
    }
}

void printVectorPerson(vector<Person> &v)
{
    for (vector<Person>::iterator it = v.begin(); it != v.end(); it++) {
        cout << (*it).name << ", " << (*it).score << endl;
    }
}

```

```
void palyGame(vector<Person> &v)
{
    // 设计随机种子
    srand(time(NULL));
    // 容器中的每个人逐一参加比赛
    for (vector<Person>::iterator it = v.begin(); it != v.end(); it++) {
        // 每位选手都要被10个评委打分
        deque<int> d;
        for (int i = 0; i < 10; i++) { // 10个评委
            int score = rand() % 41 + 60; // 60 ~ 100
            d.push_back(score);
        }

        // 对deque容器 ( 评委的10个分数 ) 排序
        sort(d.begin(), d.end());

        for (deque<int>::iterator mit = d.begin(); mit != d.end(); mit++) {
            cout << *mit << " ";
        }
        cout << endl;

        // 去掉一个最低分，一个最高分
        d.pop_front();
        d.pop_back();

        // 得到总分数
        int sum = accumulate(d.begin(), d.end(), 0);
        // 获取平均分，选手的score
        it->score = (float)sum / d.size();
    }
}

int main(int argc, char **argv)
{
    // 1. 定义一个vector容器存放Person
    vector<Person> v;
    createPerson(v);

    // 2. 遍历vector容器
    printVectorPerson(v);

    // 2. 5名选手逐一参加比赛
    palyGame(v);

    printVectorPerson(v);

    return 0;
}
```

2.4.3.5 随机数

```
#include <iostream>

using namespace std;

int main(int argc, char **argv)
{
    // 设置随机数种子，如果种子是固定的，随机数也是固定的
    // time(NULL)获取当前时间，则每次运行，随机数种子都是不一样的
    srand(time(NULL));
    for (int i = 0; i < 10; i++)
    {
        // rand() 函数的返回值就是随机数
        int num = rand();
        cout << num << " ";
    }
    cout << endl;

    return 0;
}
```

2.5 stack容器

2.5.1 stack容器基本概念

stack是一种先进后出(First In Last Out, FILO)的数据结构，它只有一个出口，形式如下图所示。stack容器允许新增元素，移除元素，取得栈顶元素，但是除了最顶端外，没有任何其他方法可以存取stack的其他元素。换言之，stack不允许有遍历行为。有元素推入栈的操作称为：push，将元素推出stack的操作称为pop。

2.5.2 stack没有迭代器

stack所有元素的进出都必须符合“先进后出”的条件，只有stack顶端的元素，才有机会被外界取用。stack不提供遍历功能，也不提供迭代器。

2.5.3 stack常用API

```
1. stack构造函数
stack<T> stk; // stack采用模板类实现，stack对象的默认构造形式：
stack(const stack &stk); // 拷贝构造函数

2. stack赋值操作
stack& operator=(const stack &stk); // 重载等号操作符

3. stack数据存取操作
push(elem); // 向栈顶添加元素
pop(); // 从栈顶移除第一个元素
top(); // 返回栈顶元素

4. stack大小操作
```

```
empty(); // 判断堆栈是否为空
size(); // 返回堆栈的大小
```

```
void test01()
{
    stack<int> s;
    // 入栈
    s.push(10);
    s.push(20);
    s.push(30);
    s.push(40);
    if (s.empty()) {
        cout << "栈为空" << endl;
    } else {
        cout << "栈不为空" << endl;
    }
    while (!s.empty()) //非空，返回false
    {
        cout << s.top() << endl;
        s.pop();
    }
}
```

2.6 queue容器

队列容器：先进先出

queue允许从一端新增元素，从另一端移除元素。

2.6.1 queue没有迭代器

queue所有元素的进出都必须符合“先进先出”的条件，只有queue的顶端元素才有机会被外界取用。queue不提供遍历功能，也不提供迭代器。

2.6.2 queue常用API

1. queue构造函数

```
queue<T> queT; // queue采用模板类实现，queue对象的默认构造形式
queue(const queue &que); // 拷贝构造函数
```

2. queue存取、插入和删除操作

```
push(elem); // 往队尾添加元素
pop(); // 从队头移除第一个元素
back(); // 返回最后一个元素
front(); // 返回第一个元素
```

3. queue赋值操作

```
queue& operator=(const queue &que); // 重载等号操作符
```

4. queue大小操作

```
empty(); // 判断队列是否为空  
size(); // 返回队列大小
```

```
void test01()  
{  
    queue<int> q;  
    q.push(10);  
    q.push(20);  
    q.push(30);  
    q.push(40);  
  
    if(q.empty()) {  
        cout << "容器为空" << endl;  
    } else {  
        cout << "容器非空" << endl;  
        cout << "size = " << q.size() << endl;  
        cout << "队头元素 = " << q.front() << endl;  
        cout << "队尾元素 = " << q.back() << endl;  
    }  
  
    cout << "遍历队列" << endl;  
    while (!q.empty()) {  
        cout << q.front() << endl;  
        q.pop();  
    }  
}
```

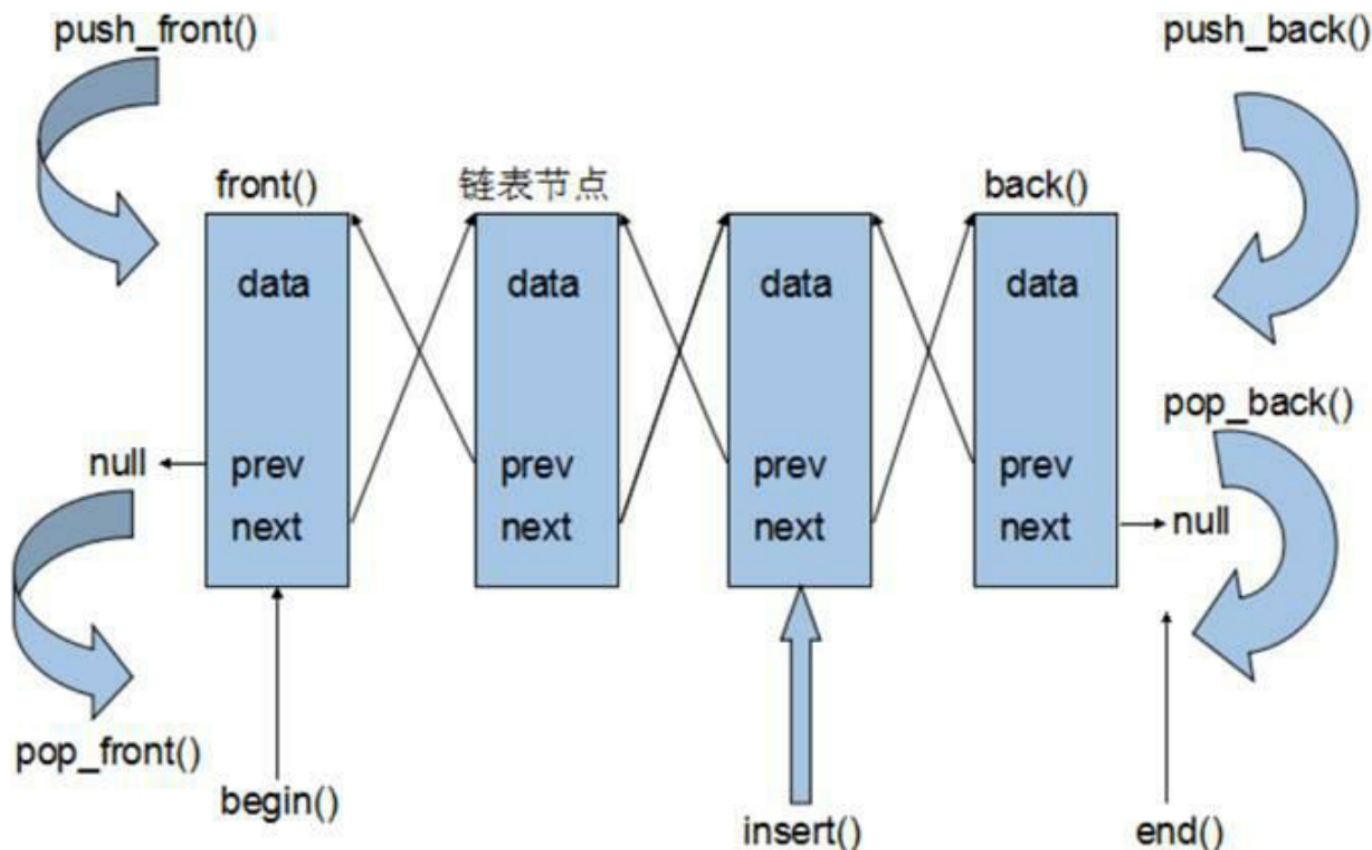
2.7 list链表容器

链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。

链表由一系列结点(链表中每一个元素称为结点)组成，结点可以在运行时动态生成。每个结点包括两个部分：一个是存储数据元素的数据域，另一个是存储下一个结点地址的指针域。相较于vector的连续线性空间，list就显得负责许多，它的好处是每次插入或者删除一个元素，就是配置或者释放一个元素的空间。因此，list对于空间的运用有绝对的精准，一点也不浪费。而且，对于任何位置的元素插入或元素的移除，list永远是常数时间。

list和vector是两个最常被使用的容器，list容器是一个**双向链表**。采用动态存储分配，不会造成内存浪费和溢出。链表执行插入和删除操作十分方便，修改指针即可，不需要移动大量元素。链表灵活，但时间和空间额外耗费较大。

list容器(双向链表)的迭代器必须具备前移、后移。提供的是双向迭代器。



回顾

- list:双向迭代器，双向链表
- vector:随机访问迭代器，单端动态数组
- deque:随机访问迭代器，双端动态数组
- stack:无迭代器，栈容器
- queue:无迭代器，队列容器

2.7.1 链表的常用API

1. list构造函数

```
list<T> lstT; // list采用模板类实现，对象的默认构造形式；
list(beg, end); // 构造函数将[beg, end)区间中的元素拷贝给本身
list(n, elem); // 构造函数将n个elem拷贝给本身
list(const list &lst); // 拷贝构造函数
```

2. list数据元素插入和删除操作

```
push_back(elem); // 在容器尾部加入一个元素
pop_back(); // 删除容器中最后一个元素
push_front(elem); // 在容器开头插入一个元素
pop_front(); // 从容器开头移除第一个元素
insert(pos, elem); // 在pos位置插入elem元素的拷贝，返回新数据的位置
insert(pos, n, elem); // 在pos位置插入n个elem数据，无返回值
insert(pos, beg, end); // 在pos位置插入[beg, end)区间的数据，无返回值
clear(); // 移除容器的所有数据
erase(beg, end); // 删除[beg, end)区间的数据，返回下一个数据的位置
erase(pos); // 删除pos位置的数据，返回下一个数据的位置
remove(elem); // 删除容器中所有与elem相匹配的元素。
```

3. list大小操作

size(); // 返回容器中元素的个数

empty(); // 判断容器是否为空

resize(num); // 重新指定容器的长度为num，若容器变长，则以默认值填充新位置。如果容器变短，则末尾超出容器长度的元素被删除。

resize(num, elem); // 重新指定容器的长度为num，若容器变长，则以elem值填充新位置。

4. list赋值操作

assign(beg, end); // 将[beg, end)区间中的数据拷贝赋值给本身。

assign(n, elem); // 将n个elem拷贝赋值给本身

list& operator=(const list &lst); // 重载等号操作符

swap(lst); // 将lst与本身的元素互换

5. list数据的存取

front(); // 返回第一个元素

back(); // 返回最后一个元素、

6. list反转排序

reverse(); // 反转链表

sort(); // list排序

```

void test01()
{
    list<int> l;
    l.push_back(10);
    l.push_back(20);
    l.push_back(30);
    l.push_back(40);

    printListInt(l); // 10 20 30 40

    // insert(pos, n, elem);
    // 迭代器+n，只有随机访问迭代器支持
    // 而list容器的迭代器是双向迭代器，不支持+n
    // l.insert(l.begin() + 2, 3, 100); // error: no match for 'operator+'
    // (operand types are 'std::__cxx11::list<int>::iterator' {aka
    // 'std::_List_iterator<int>'} and 'int')
    list<int>::iterator it = l.begin();
    // ++ 随机访问迭代器以及双向迭代器都支持
    it++;
    it++; // 两次++不等于+2，因为重载运算符++
    l.insert(it, 3, 100);
    printListInt(l); // 10 20 100 100 100 30 40

    // remove(elem); // 删除容器中所有与elem值匹配的元素
    l.remove(100);
    printListInt(l); // 10 20 30 40

    // 链表反转
    l.reverse();
    printListInt(l); // 40 30 20 10

```



```

// list容器的排序使用list类本身的排序算法
// sort是系统提供的蒜贩，仅支持随机访问迭代器(不支持list)
// list不能使用系统算法
// sort(l.begin(), l.end()); // error: no match for 'operator-' (operand types
are 'std::_List_iterator<int>' and 'std::_List_iterator<int>')
l.sort();
printListInt(l); // 10 20 30 40
}

```

2.7.2 list容器存放自定义数据

如果删除某个结点必须重载==运算符,remove()

```

#include <vector>
#include <algorithm>
#include <string>
#include <iostream>
#include <list>

using namespace std;

class Person
{
public:
    string name;
    int age;
public:
    Person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }
    bool operator==(const Person &ob)
    {
        if (this->age == ob.age && this->name == ob.name) {
            return true;
        }
        return false;
    }
};

void printListPerson(list<Person> &l)
{
    for (list<Person>::iterator it = l.begin(); it != l.end(); it++) {
        cout << it->name << ", " << it->age << endl;
    }
}

void test01()
{

```

```
// 存放自定义的数据
list<Person> l;
l.push_back(Person("德玛西亚", 18));
l.push_back(Person("提莫", 28));
l.push_back(Person("狗头", 38));
l.push_back(Person("牛头", 19));
printListPerson(l);

// 删除狗头
Person tmp("狗头", 38);
l.remove(tmp); // error: no match for 'operator==' (operand types are 'Person'
and 'const value_type' {aka 'const Person'})
printListPerson(l);
}
```

2.7.3 list容器自定义数据排序 · 必须实现排序规则

1. 重载<运算符

```
#include <vector>
#include <algorithm>
#include <string>
#include <iostream>
#include <list>

using namespace std;

class Person
{
public:
    string name;
    int age;
public:
    Person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }
    // 重载<运算符
    bool operator<(const Person &ob)
    {
        return this->age < ob.age;
    }
};

void printListPerson(list<Person> &l)
{
    for (list<Person>::iterator it = l.begin(); it != l.end(); it++) {
        cout << it->name << ", " << it->age << endl;
    }
}
```

```
void test02()
{
    list<Person> l;
    l.push_back(Person("德玛西亚", 18));
    l.push_back(Person("提莫", 28));
    l.push_back(Person("狗头", 38));
    l.push_back(Person("牛头", 19));
    printListPerson(l);

    // 对于自定义数据，可以重载<运算符
    l.sort(); // error: no match for 'operator<' (operand types are 'Person' and
    'Person')
    printListPerson(l);
}
```

list对自定义数据排序指定排序规则

```
bool myComparePerson(const Person &ob1, const Person &ob2)
{
    return ob1.age < ob2.age;
}

void test03()
{
    list<Person> l;
    l.push_back(Person("德玛西亚", 18));
    l.push_back(Person("提莫", 28));
    l.push_back(Person("狗头", 38));
    l.push_back(Person("牛头", 19));
    printListPerson(l);
    cout << "=====" << endl;
    l.sort(myComparePerson);
    printListPerson(l);
}
```

2.7.4 vector对自定义数据排序指定排序规则

方法一：普通函数实现排序规则

```
void test04()
{
    vector<Person> v;
    v.push_back(Person("德玛西亚", 18));
    v.push_back(Person("提莫", 28));
    v.push_back(Person("狗头", 38));
    v.push_back(Person("牛头", 19));
    printVectorPerson(v);

    // 默认比较方式从小到大，对于自定义数据没有排序规则
}
```

```
// vector存放自定义数据，可指定排序规则
sort(v.begin(), v.end(), myComparePerson);
printVectorPerson(v);
}
```

2.7.5 仿函数指定排序规则

```
// 仿函数指定排序规则
class MyComparePerson
{
public:
    bool operator()(const Person &ob1, const Person &ob2)
    {
        return ob1.age < ob2.age;
    }
};

void test05()
{
    vector<Person> v;
    v.push_back(Person("德玛西亚", 18));
    v.push_back(Person("提莫", 28));
    v.push_back(Person("狗头", 38));
    v.push_back(Person("牛头", 19));
    printVectorPerson(v);

    sort(v.begin(), v.end(), MyComparePerson());
    printVectorPerson(v);
}
```

2.7.6 仿函数回顾

```
class MyAdd
{
public:
    // 本质是成员函数，函数名叫operator()
    int operator()(int a, int b)
    {
        cout << "调用了operator() int int" << endl;
        return a + b;
    }
    int operator()(int a, int b, int c)
    {
        cout << "调用了operator() int int int" << endl;
        return a + b + c;
    }
};

void test06()
{
    MyAdd ob1;
```

```

int ret = ob1.operator()(10, 20);
cout << ret << endl; // 30
// 严格意义：ob1是对象和()结合，调用operator()成员函数
// ob1(100,200) ob1本质是对象不是函数名，只是形式像函数调用
// 叫作仿函数
cout << ob1(20, 50) << endl; // 70
cout << MyAdd()(40, 90) << endl; // 130

cout << ob1(10, 20, 30) << endl; // 60

cout << MyAdd()(1000, 2000) << endl; // 3000，产生一个匿名对象
}

```

输出：

```

调用了operator() int int
30
调用了operator() int int
70
调用了operator() int int
130
调用了operator() int int int
60
调用了operator() int int
3000

```

2.8 lambda表达式

```

void myPrintVectorInt(vector<int> &v)
{
    for (vector<int>::iterator it = v.begin(); it != v.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

// 指定打印方式
void myPrint01(int val)
{
    cout << val << " ";
}

void test07()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);
}

```

```
// 方法一：访问v容器，普通函数
myPrintVectorInt(v);

// 方法二：访问v容器，系统算法for_each
for_each(v.begin(), v.end(), myPrint01);
cout << endl;

// 方法三：访问v容器 lambda表达式
// []标识函数名 ()参数列表 {}函数体
for_each(v.begin(), v.end(), [](int val){ cout << val << " "; });
cout << endl;
}
```

2.9 set容器

set的特性是所有元素都会根据元素的键值自动被排序，set元素不像map那样可以同时拥有实值和键值，set的元素既是键值又是实值。set不允许两个元素有相同的键值。

可以通过set的迭代器改变set元素的值吗？不行，因为set元素值就是其键值，关系到set的排序规则。如果任意改变set元素值，会严重破坏set组织。换句话说，set的iterator是一种const iterator。set拥有和list某些相同的性质，当对容器中的元素进行插入操作或者删除操作的时候，操作之前所有的迭代器，在操作完成之后依然有效，被删除的那个元素的迭代器必然是一个例外。

multiset特性及用法和set完全相同，唯一的差别在于它允许键值重复。set和multiset的底层实现是红黑树，红黑树为平衡二叉树的一种。

树的简单知识：二叉树就是任何节点最多只允许有两个子节点，分别是左子节点和右子节点。

set容器特点：

所有元素都会根据元素的键值自动被排序。

2.9.1 set容器常用API

```
1. set构造函数
set<T> st; // set容器默认构造函数
multiset<T> mst; // multiset容器默认构造函数
set(const set &st); // 拷贝构造函数

2. set赋值操作
set& operator=(const set &st); // 重载等号操作符
swap(st); // 交换两个集合容器

3. set大小操作
size(); // 返回容器中元素的数目
empty(); // 判断容器是否为空

4. set插入和删除操作
insert(elem); // 在容器中插入元素
clear(); // 清除所有元素
```

```
erase(pos); // 删除pos迭代器所指的元素，返回下一个元素的迭代器
erase(beg, end); // 删除区间[beg, end)的所有元素，返回下一个元素的迭代器
erase(elem); // 删除容器中值为elem的元素
```

```
void test01()
{
    set<int> s;
    s.insert(30);
    s.insert(10);
    s.insert(20);
    s.insert(50);
    s.insert(40);

    for_each(s.begin(), s.end(), [](int val){ cout << val << " "; }); // 10 20 30
40 50
    cout << endl;

    set<int>::iterator it = s.begin();
    // set容器提供的是只读迭代器const_iterator
    // 迭代器不允许修改值
    // *it = 100; // error: assignment of read-only location
    'it.std::_Rb_tree_const_iterator<int>::operator*()'

    cout << "size = " << s.size() << endl; // size = 5

    s.erase(s.begin()); // 删除起始位置
    for_each(s.begin(), s.end(), [](int val){ cout << val << " "; }); // 20 30 40
50
    cout << endl;

    s.erase(40); // 根据元素删除
    for_each(s.begin(), s.end(), [](int val){ cout << val << " "; }); // 20 30 50
    cout << endl;
}
```

2.9.2 set查找操作

```
find(key); // 查找key是否存在，若存在，返回该键的元素的迭代器；若不存在，返回set.end();
count(key); // 查找键key的元素个数
```

```
void test02()
{
    set<int> s;
    s.insert(30);
    s.insert(10);
    s.insert(20);
```

```

s.insert(50);
s.insert(40);

// find(key); // 查找key是否存在，若存在，返回该键的元素的迭代器；若不存在，返回
set.end();
set<int>::const_iterator it = s.find(20);
if (it == s.end()) {
    cout << "没有找到" << endl;
} else {
    cout << "找到" << *it << endl;
}

// count(key); // 查找键key的元素个数
// set容器的键值是不重复的，那么count(key)只能是0或1。
int cnt = s.count(30);
cout << "cnt = " << cnt << endl; // cnt = 1
cout << s.count(200) << endl; // 0
}

```

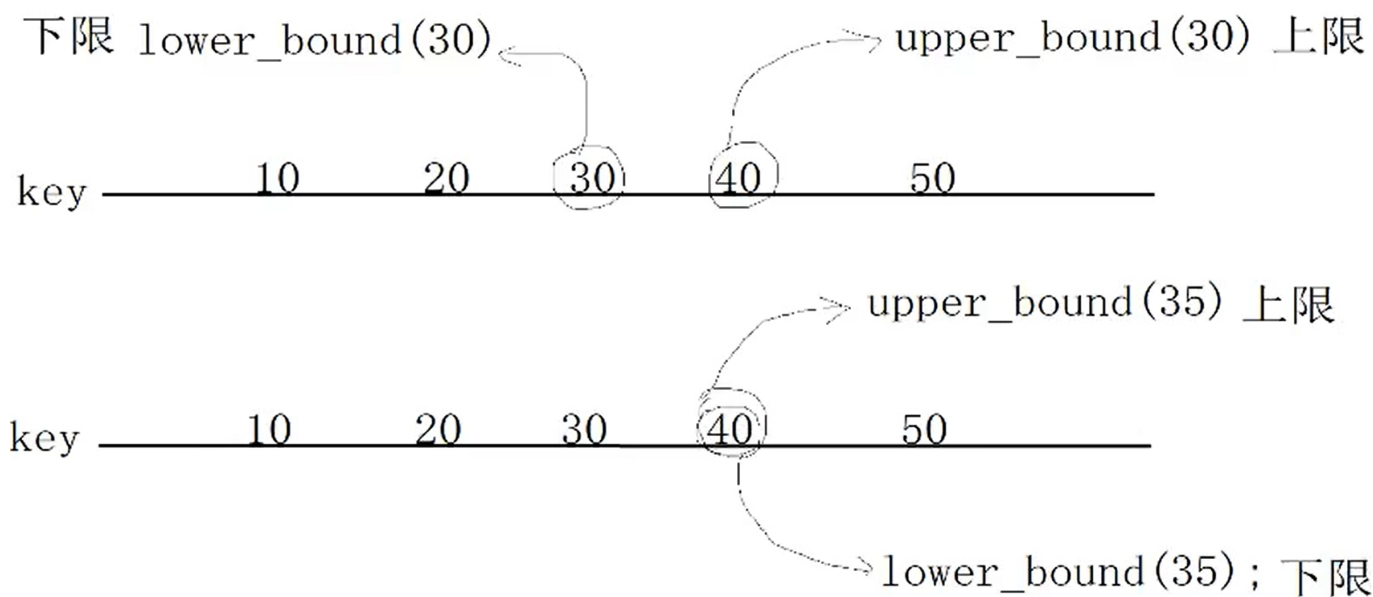
2.9.3 set上、下限查找

lower_bound(keyElem); // 返回第一个key>=keyElem元素的迭代器
upper_bound(keyElem); // 返回第一个key>keyElem元素的迭代器。

```

1 lower_bound(keyElem); // 返回第一个key>=keyElem元素的迭代器。
2 upper_bound(keyElem); // 返回第一个key>keyElem元素的迭代器。

```



```

void test03()
{
    set<int> s;
    s.insert(30);
}

```



```

s.insert(10);
s.insert(20);
s.insert(50);
s.insert(40);

for_each(s.begin(), s.end(), [](int val) { cout << val << " "; }); // 10 20 30
40 50
cout << endl;
// lower_bound(keyElem); // 返回第一个key>=keyElem元素的迭代器，下限
set<int>::const_iterator it = s.lower_bound(30);
if (it != s.end()) {
    cout << "下限查找:" << *it << endl; // 下限查找:30，下限查找:30
} else {
    cout << "下限查找:不存在" << endl;
}
// upper_bound(keyElem); // 返回第一个key>keyElem元素的迭代器，上限
it = s.upper_bound(30);
if (it != s.end()) {
    cout << "上限查找:" << *it << endl; // 下限查找:30，上限查找:40
} else {
    cout << "上限查找:不存在" << endl;
}
}

```

2.9.4 set equal_ubound查找

`equal_range(keyElem);` // 返回容器中key与keyElem相等的上下限的两个迭代器。

```

void test04()
{
    set<int> s;
    s.insert(30);
    s.insert(10);
    s.insert(20);
    s.insert(50);
    s.insert(40);

    for_each(s.begin(), s.end(), [](int val) { cout << val << " "; }); // 10 20 30
40 50
    cout << endl;

    // equal_range(keyElem); // 返回容器中key与keyElem相等的上下限的两个迭代器。
    // equal_range 返回的是对组
    // first对组中第一个值，second对组中第二个值
    pair<set<int>::const_iterator, set<int>::const_iterator> pit =
s.equal_range(30);
    if (pit.first != s.end()) {
        cout << "下限查找:" << *(pit.first) << endl; // 下限查找:30
    } else {

```

```

        cout << "下限查找:不存在" << endl;
    }

    if (pit.second != s.end()) {
        cout << "上限查找:" << *(pit.second) << endl; // 上限查找:40
    } else {
        cout << "上限查找:不存在" << endl;
    }
}

```

2.9.5 对组 pair

对组(pair)将一对值组成成一个值，这一对值可以具有不同的数据类型，两个值可以分别用pair的两个公有属性first和second访问。类模板：`template<class T1, class T2> struct pair。`

```

void test05()
{
    // 9527--星爷 10086--移动 10010--联通
    // 定义对组方式1：
    pair<int, string> pair1(9527, "星爷");
    cout << "编号：" << pair1.first << "，人物：" << pair1.second << endl; // 编号：9527，人物：星爷

    // 定义对组方式2：(推荐)
    pair<int, string> pair2 = make_pair(10086, "移动");
    cout << "编号：" << pair2.first << "，人物：" << pair2.second << endl; // 编号：10086，人物：移动

    // pair=赋值
    pair<int, string> pair3 = pair2;
    cout << "编号：" << pair2.first << "，人物：" << pair2.second << endl; // 编号：10086，人物：移动
}

```

2.9.6 set更改容器的默认排序规则(推荐使用仿函数)

```

class MyGreater
{
public:
    bool operator()(int val1, int val2)
    {
        return val1 > val2;
    }
};

void test06()
{
    // 默认从小到大排序
    // 改成从大到小排序,一旦插入过后无法修改排序规则
}

```

```

// set<int, 排序规则> s;
// set<int, std::greater<int>> s; // 方法一
set<int, MyGreater> s; // 方法二,自定义排序规则
s.insert(30);
s.insert(10);
s.insert(20);
s.insert(50);
s.insert(40);
for_each(s.begin(), s.end(), [](int val) { cout << val << " "; }); // 50 40 30
20 10
cout << endl;

}

```

2.9.7 set容器自定义数据必须指定排序规则,重载<运算符

```

class Person
{
public:
    string name;
    int age;
    Person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }

    // 方法一:重载<运算符 从小到大
    bool operator<(const Person &ob) const
    {
        return this->age < ob.age;
    }
};

void test07()
{
    set<Person> s;
    // set存放自定义数据,需提供排序规则,否则无法在内存中进行顺序存放
    s.insert(Person("德玛西亚", 18)); // error: no match for 'operator<' (operand
types are 'const Person' and 'const Person')
    s.insert(Person("冯楠", 21));
    s.insert(Person("小炮", 20));
    s.insert(Person("小法", 19));

    for_each(s.begin(), s.end(), [](const Person &ob){ cout << "姓名:" << ob.name
<< ", 年龄:" << ob.age << endl;});
}

```

运行结果:

姓名:德玛西亚, 年龄:18
姓名:小法, 年龄:19
姓名:小炮, 年龄:20
姓名:冯楠, 年龄:21

如果要从大到小排序,就得重载>运算符.有些繁琐,可以指定排序规则,如下:

2.9.7 set容器自定义数据必须指定排序规则,指定排序规则

```
class MyGreaterPerson
{
public:
    bool operator()(const Person &ob1, const Person &ob2)
    {
        return ob1.age > ob2.age;
    }
};

void test08()
{
    set<Person, MyGreaterPerson> s;
    s.insert(Person("德玛西亚", 18));
    s.insert(Person("冯楠", 21));
    s.insert(Person("小炮", 20));
    s.insert(Person("小法", 19));
    for_each(s.begin(), s.end(), [](const Person &ob){ cout << "姓名:" << ob.name
<< ", 年龄:" << ob.age << endl;});
}
```

运行结果:

姓名:冯楠, 年龄:21
姓名:小炮, 年龄:20
姓名:小法, 年龄:19
姓名:德玛西亚, 年龄:18

2.9.8 multiset容器

multiset容器可以插入重复的值.

set容器如果插入重复的值,后面的插入会无效.

```
void test09()
{
    set<int> s;
    pair<set<int>::const_iterator, bool> it1;
```

```
pair<set<int>::const_iterator, bool> it2;
it1 = s.insert(10);
it2 = s.insert(10);
if (it1.second == true) {
    cout << "第一次插入成功" << endl; // 第一次插入成功
} else {
    cout << "第一次插入失败" << endl;
}

if (it2.second == true) {
    cout << "第二次插入成功" << endl;
} else {
    cout << "第二次插入失败" << endl; // 第二次插入失败
}
}
```

multiset可以插入相同的元素

```
void test10()
{
    multiset<int> mst;
    mst.insert(10);
    mst.insert(10);
    for_each(mst.begin(), mst.end(), [](int val){ cout << val << " ";}); // 10 10,
    两个相同元素插入成功
    cout << endl;
}
```

2.10 map容器

map的特性是所有元素会根据元素的键值自动排序，map所有元素都是pair，同时拥有键值和实值。

map容器的键值不可变，但实值可以修改。

2.10.1 map容器的常用API

```
1. map构造函数
map<T1, T2> mapTT; // map默认构造函数
map(const map &mp); // 拷贝构造函数

2. map赋值操作
map& operator=(const map &mp); // 重载等号操作符
swap(mp); // 交换两个集合容器

3. map大小操作
size(); // 返回容器中元素的数目
empty(); // 判断容器是否为空
```

4. map插入数据元素操作

```

insert(...); // 往容器插入元素·返回pair<iterator, bool>
map<int, string> mapStu;
// 第一种 通过pair的方式插入对象
mapStu.insert(pair<int, string>(3, "小张"));
// 第二种 通过pair的方式插入对象
mapStu.insert(make_pair(-1, "小张"));
// 第三种 通过value_type的方式插入对象
mapStu.insert(map<int, string>::value_type(2, "小陈"));
// 第四种 通过数组的方式插入值
mapStu[3] = "小刘";
mapStu[5] = "小王";

```

5. map删除操作

```

clear(); // 删除所有元素
erase(pos); // 删除pos迭代器所指的元素·返回下一个元素的迭代器
erase(beg, end); // 删除区间[beg, end)的所有元素·返回下一个元素的迭代器
erase(keyElem); // 删除容器中key为keyElem的对组

```

6. map查找操作

```

find(key); // 查找key是否存在·若存在·返回该键的元素的迭代器；若不存在·返回map.end()
count(keyElem); // 返回容器中key为keyElem的对组个数。对map来说·要么是0·要么是1
lower_bound(keyElem); // 返回第一个key>=keyElem元素的迭代器
upper_bound(keyElem); // 返回第一个key>keyElem元素的迭代器
equal_bound(keyElem); // 返回容器中key与keyElem相等的上下限的两个迭代器

```

```

void test01()
{
    // 9527--"星爷" 10086--"移动" 10010--"联通"
    // int为键值的类型·string为实值的类型
    map<int, string> mp;
    // 第一种：
    mp.insert(pair<int, string>(9527, "星爷"));
    // 第二种：(推荐)
    mp.insert(make_pair(10086, "移动"));
    // 第三种：使用value_type
    mp.insert(map<int, string>::value_type(10010, "联通"));
    // 第四种：读map容器数据的时候·推荐
    mp[10000] = "电信";
    // mp[10] 寻找key为10的实值
    // 如果map容器中没有key为10·使用mp[10]会创建一个key值为10·实值为空的对组。
    // 如果容器中有key为10·那么mp[10]代表key=10的实值。
    // 建议：10是存在的
    // cout << mp[10] << endl;

    // 遍历
    for_each(mp.begin(), mp.end(), [](pair<int, string> p){cout << p.first << ":"
    << p.second << endl;});

    // 只想查看key==9527的实值·保证key是存在的
    cout << mp[9527] << endl;
}

```

```

    cout << mp[10010] << endl;

    // 如果不能确定key值是否存在
    map<int, string>::const_iterator it = mp.find(10086);
    if (it == mp.end()) {
        cout << "未找到相关节点" << endl;
    } else {
        cout << "找到相关节点: " << it->second << endl; // 找到相关节点: 移动
    }
}

```

2.10.2 map和vector容器配合使用

2.10.2.1 LOL职业联赛：有4个战队，随机抽签出场，请打印出场顺序

```

void test02()
{
    // 设置种子
    srand(time(NULL));
    // 战队容器 ( 战队编号, 战队名称 )
    map<int, string> m;
    m.insert(make_pair(1, "RNG"));
    m.insert(make_pair(2, "IG"));
    m.insert(make_pair(3, "WE"));
    m.insert(make_pair(4, "EDG"));

    // 使用vector存放战队编号
    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    v.push_back(4);

    // 随机抽签 ( 洗牌 )
    random_shuffle(v.begin(), v.end());

    // 随机出场
    for_each(v.begin(), v.end(), [&m] (int key){
        cout << m[key] << "战队出场了" << endl;
    });
}

```

2.10.2.2 5名员工加入3个部门

```

class Person
{
public:
    string name;
}

```

```

    int age;
    Person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }
};

void createVectorPerson(vector<Person> &v)
{
    v.push_back(Person("员工A", 21));
    v.push_back(Person("员工B", 23));
    v.push_back(Person("员工C", 21));
    v.push_back(Person("员工D", 22));
    v.push_back(Person("员工E", 21));
}

void PersonByGroup(vector<Person> &v, multimap<int, Person> &m)
{
    // 逐个将员工分配到各个部门内
    for(vector<Person>::iterator it = v.begin(); it != v.end(); it++) {
        cout << "请输入<" << it->name << ">将要加入的部门:1 2 3" << endl;
        int dep = 0;
        cin >> dep;
        if (dep >= 1 && dep <= 3) {
            m.insert(make_pair(dep, *it));
        }
    }
}

void showPersonList(multimap<int, Person> &m, int dep)
{
    switch (dep)
    {
    case 1:
        cout << "研发部:" << endl;
        break;
    case 2:
        cout << "测试部:" << endl;
        break;
    case 3:
        cout << "人事部:" << endl;
        break;
    default:
        cout << "请输入正确的部门编号:1 2 3" << endl;
        break;
    }

    // 注意:m中存放的数据《部门号、员工信息》
    // 统计部门号的元素个数
    int n = m.count(dep);
    cout << "部门的人数个数: " << n << endl;
    // 由于multimap将key自动排序,重复相同key一定挨在一起
    multimap<int, Person>::const_iterator it = m.find(dep);
    if (it != m.end()) {
        for (int i = 0; i < n; i++, it++) {
            cout << "name = " << it->second.name << ", age = " << it->second.age

```



```
<< endl;
    }
}
}
void test04()
{
    // 使用vector容器将员工信息存储起来
    vector<Person> v;
    // 创建5名员工
    createVectorPerson(v);

    // 将员工分配到不同的部门 ( 员工<部门号, 员工信息>)
    multimap<int, Person> m; // 存放员工<部门号, 员工信息>
    PersonByGroup(v, m);

    // 按部门查看员工信息
    showPersonList(m, 1);
    showPersonList(m, 2);
    showPersonList(m, 3);
}
```

STL容器使用时机

	vector	deque	list	set	multiset	map	multimap
典型内存结构	单端数组	双端数组	双向链表	二叉树	二叉树	二叉树	二叉树
可随机存取	是	是	否	否	否	对key而言:不是	否
	vector	deque	list	set	multiset	map	multimap
元素搜索速度	慢	慢	非常慢	快	快	对key而言:快	对key而言:快
元素安插移除	尾端	头尾两端	任何位置	-	-	-	-

vector的使用场景：软件历史操作记录的存储，不删除，只记录。

deque的使用场景：排队购票系统。

list使用场景：公交车乘客的存储，随时可能有乘客下车，支持频繁的不确定位置元素的移除插入；

set使用场景：手机游戏个人得分记录的存储，存储要求从高分到低分分的顺序排序；

map使用场景：根据ID号存储十万个用户，想要快速通过ID查找对应的用户。

三、算法

3.1 函数对象

重载函数调用操作符的类，其对象常称为函数对象(function object)，即它们是行为类似函数的对象，也叫仿函数(functor)，其实就是重载()操作符，使得类对象可以像函数那样调用。

注意：

1. 函数对象(仿函数)是一个类，不是一个函数。
2. 函数对象(仿函数)重载了`()`操作符使得它可以像函数一样调用。

分类：假定某个类有一个重载的`operator()`，而且重载的`operator()`要求获取一个参数，我们就将这个类称为“一元仿函数”(unary functor)；相反，如果重载的`operator()`要求获取两个参数，就将这个类称为“二元仿函数”(binary functor)。

函数对象的作用主要是什么？

STL提供的算法往往都有两个版本，其中一个版本表现出最常用的某种运算，另一版本则允许用户通过`template`参数的形式来指定所要采取的策略。

```
// 函数对象是重载了函数调用符号的类
class MyPrint
{
public:
    MyPrint()
    {
        m_Num= 0;
    }
    int m_Num;
public:
    void operator()(int num)
    {
        cout << num << endl;
        m_Num++;
    }
}

// 函数对象
// 重载了()操作符的类实例化的对象，可以像普通函数那样调用，可以有参数，可以有返回值
void test01()
{
    MyPrint myPrint;
    myPrint(20);
}

// 函数对象超出了普通函数的概念，可以保存函数的调用状态
void test02()
{
    MyPrint myPrint;
    myPrint(20);
    myPrint(20);
    myPrint(20);
    cout << myPrint.m_Num << endl;
}

void doBusiness(MyPrint print, int num)
{
    print(num);
}
```

```
// 函数对象作为参数
void test03()
{
    // 参数1：匿名函数对象
    doBusiness(MyPrint(), 30);
}
```

总结：

1. 函数对象通常不定义构造函数和析构函数，所以在构造和析构时不会发生任何问题，避免了函数调用的运行时问题。
2. 函数对象超出普通函数的概念，函数对象可以有自己的状态(可以有类成员)。
3. 函数对象可内联编译，性能好。用函数指针几乎不可能。
4. 模板函数对象使函数对象具有通用性，这也是它的优势之一。

3.2 谓词

谓词是指普通函数或重载的`operator()`返回值是`bool`类型的函数对象(仿函数)。如果`operator`接受一个参数，那么叫作一元谓词，如果接受两个参数，那么叫作二元谓词，谓词可作为一个判断式。

3.2.1 一元谓词

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// 普通函数作为一元谓词
bool greaterThan20(int val)
{
    return val > 20;
}
// 使用仿函数作为一元谓词
class MyGreaterThan20
{
public:
    bool operator()(int val)
    {
        return val > 20;
    }
};
int main(int argc, char **argv)
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);
    v.push_back(50);

    for_each(v.begin(), v.end(), [](int val){cout << val << " ";}); // 10 20 30 40
```

```

50
    cout << endl;

    // 需求：找出第一个大于20的数
    // 方法一：使用普通函数作为一元谓词
    vector<int>::iterator it = find_if(v.begin(), v.end(), greaterThan20);
    if (it != v.end()) {
        cout << "第一个大于20的数为：" << *it << endl;
    }
    // 方法二：使用仿函数作为一元谓词
    it = find_if(v.begin(), v.end(), MyGreaterThan20());
    if (it != v.end()) {
        cout << "第一个大于20的数为：" << *it << endl;
    }

    return 0;
}

```

输出：

```

10 20 30 40 50
第一个大于20的数为：30
第一个大于20的数为：30

```

3.2.2 二元谓词

```

// 普通函数作为二元谓词
bool myGreaser(int v1, int v2)
{
    return v1 > v2;
}
// 仿函数作为二元谓词
class MyGreater
{
public:
    bool operator()(int a, int b)
    {
        return a > b;
    }
};
void test02()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);
    v.push_back(50);
}

```

```

    for_each(v.begin(), v.end(), [](int val){cout << val << " ";}); // 10 20 30 40
50
    cout << endl;

    // 从大到小排序
    // 方法一：使用普通函数作为二元谓词
    // sort(v.begin(), v.end(), myGreaser);
    // 方法二：使用仿函数作为二元谓词
    sort(v.begin(), v.end(), MyGreater());
    for_each(v.begin(), v.end(), [](int val){cout << val << " ";}); // 50 40 30 20
10
    cout << endl;
}

```

3.3 内建函数对象

STL内建了一些函数对象。分为：算法类函数对象，关系运算类函数对象，逻辑运算类仿函数。这些仿函数所产生的对象，用法和一般函数完全相同，当然我们还可以产生无名的临时对象来履行函数功能。使用内建函数对象，需要引入头文件`#include<algorithm>`

6个算法类函数对象

除了`negate`是一元运算，其他都是二元运算。

```

template<class T> T plus<T> // 加法仿函数
template<class T> T minus<T> // 减法仿函数
template<class T> T multiplies<T> // 乘法仿函数
template<class T> T divides<T> // 除法仿函数
template<class T> T modulus<T> // 取模仿函数
template<class T> T negate<T> // 取反仿函数

```

6个关系运算类函数对象

每一种都是二元运算。

```

template<class T> bool equal_to<T> // 等于
template<class T> bool not_equal_to<T> // 不等于
template<class T> bool greater<T> // 大于
template<class T> bool greater_equal<T> // 大于等于
template<class T> bool less<T> // 小于
template<class T> bool less_equal<T> // 小于等于

```

逻辑类运算函数

`not`为一元运算，其余为二元运算

```
template<class T> bool logical_and<T> // 逻辑与
template<class T> bool logical_or<T> // 逻辑或
template<class T> bool logical_not<T> // 逻辑非
```

```
void test03()
{
    plus<string> p;
    cout << p("qwer", "tyuiop") << endl; // qwertyuiop

    minus<int> m;
    cout << m(90, 70) << endl; // 20

    multiplies<float> mp;
    cout << mp(30.9, 20.1) << endl; // 621.09

    divides<float> d;
    cout << d(90, 1.2) << endl; // 75

    modulus<int> mo;
    cout << mo(1000, 7) << endl; // 6

    negate<int> n;
    cout << n(50) << endl; // -50
}

void test04()
{
    vector<int> v;
    srand(time(NULL));
    for (int i = 0; i < 10; i++) {
        v.push_back(rand() % 100);
    }

    for_each(v.begin(), v.end(), [](int val){ cout << val << " ";}); // 37 93 89
57 22 44 39 71 53 62
    cout << endl;

    sort(v.begin(), v.end(), greater<int>());
    for_each(v.begin(), v.end(), [](int val){ cout << val << " ";}); // 93 89 71
62 57 53 44 39 37 22
    cout << endl;

    sort(v.begin(), v.end(), less_equal<int>());
    for_each(v.begin(), v.end(), [](int val){ cout << val << " ";}); // 22 37 39
44 53 57 62 71 89 93
    cout << endl;

    cout << std::boolalpha << equal_to<int>()(10, 10) << endl; // true
    cout << not_equal_to<int>()(30, 40) << endl; // true
}
```

3.4 函数适配器

扩展函数的参数接口 (假如函数有一个参数, 再扩展一个接口, 就可以传递两个参数)

3.4.1 绑定适配器

bind1st和bind2nd。

```
void myPrint(int val, int tmp)
{
    cout << val + tmp << " ";
}
// 适配器2: 公共继承binary_function
// 适配器3: 参数的萃取, <int, int, void>
// 适配器4: 对operator()进行const修饰
class MyPrint : public binary_function<int, int, void>
{
public:
    void operator()(int val, int tmp) const
    {
        cout << val + tmp << " ";
    }
};
void test01()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);
    v.push_back(50);

    // for_each(v.begin(), v.end(), myPrint); // error: too few arguments to
function
    // 出错的原因: 需求是打印的时候加上一个动态传入的值, 但是for_each读取的v中每个元素都
    只有一个参数, 而myPrint希望传入两个参数。
    // 修改: 使用函数适配器, 扩展函数参数
    // for_each(v.begin(), v.end(), bind2nd(myPrint, 10)); // error: 'void(int,
int)' is not a class, struct, or union type
    // bind2nd只支持类、结构体或者union类型

    // 适配器1: bind2nd或bind2st绑定参数
    for_each(v.begin(), v.end(), bind2nd(MyPrint(), 20)); // error: no type named
'first_argument_type' in 'class MyPrint'
    // 这个类必须继承binary_function
    // bind2nd将参数绑定到第二个参数位置
    cout << endl;
    for_each(v.begin(), v.end(), bind1st(MyPrint(), 40));
    // bind1st将参数绑定到第一个参数位置
}
```

运行结果：

```
val = 10, tmp = 20, sum = 30
val = 20, tmp = 20, sum = 40
val = 30, tmp = 20, sum = 50
val = 40, tmp = 20, sum = 60
val = 50, tmp = 20, sum = 70

val = 40, tmp = 10, sum = 50
val = 40, tmp = 20, sum = 60
val = 40, tmp = 30, sum = 70
val = 40, tmp = 40, sum = 80
val = 40, tmp = 50, sum = 90
```

使用函数绑定适配器的步骤

1. bind2nd或bind2st绑定参数；
2. 公共继承binary_function；
3. 参数的萃取 · <int, int, void> ；
4. 对operator()进行const修饰。

总结：

函数适配器bind1st bind2nd，如果想使用绑定适配器，需要我们自己的函数对象继承binary_function或者unary_function(二元仿函数，继承binary_function；一元仿函数，继承unary_function)。

bind1st和bind2nd区别？

bind1st：将参数绑定为函数对象的第一个参数；

bind2nd：将参数绑定为函数对象的第二个参数；

bind1st和bind2nd将二元函数对象转为一元函数对象。

3.4.2 取反适配器

取反适配器通常针对一元谓词和二元谓词。

1. not1取反适配器

针对只有一个参数的函数对象。

```
class MyGreaterThan3 : public unary_function<int, bool>
{
public:
    // 一元谓词
    bool operator()(int val) const
    {
        return val > 3;
    }
};

void test02()
```



```

{
    vector<int> v;
    for (int i = 0; i < 10; i++)
    {
        v.push_back(i);
    }

    // 找出第一个大于3的数
    vector<int>::iterator it;
    it = find_if(v.begin(), v.end(), MyGreaterThan3());
    if (it != v.end()) {
        cout << "*it = " << *it << endl; // 4
    }

    // 找出第一个小于等于3的数
    it = find_if(v.begin(), v.end(), not1(MyGreaterThan3())); // error: no type
    named 'argument_type' in 'class MyGreaterThan3'
    if (it != v.end()) {
        cout << "*it = " << *it << endl; // 0
    }
}

```

函数对象同样需要公共继承自 `unary_function`

2. `not2`取反适配器

针对有两个参数的适配器

```

class MyGreaterInt : public binary_function<int, int, bool>
{
public:
    bool operator()(int v1, int v2) const
    {
        return v1 > v2;
    }
};

void test03()
{
    vector<int> v;
    v.push_back(2);
    v.push_back(5);
    v.push_back(1);
    v.push_back(4);
    v.push_back(3);

    for_each(v.begin(), v.end(), [](int v) { cout << v << " "; }); // 2 5 1 4 3
    cout << endl;

    // 默认从小到大
    sort(v.begin(), v.end()); // 1 2 3 4 5
    // 更改排序规则，从大到小
    sort(v.begin(), v.end(), MyGreaterInt()); // 5 4 3 2 1
}

```

```
// 使用not2对MyGreaterInt()取反·从小到大
sort(v.begin(), v.end(), not2(MyGreaterInt())); // 1 2 3 4 5
// 使用not2对内联函数取反
sort(v.begin(), v.end(), not2(greater<int>())); // 1 2 3 4 5
// 也可以直接使用内联函数实现从小到大排序
sort(v.begin(), v.end(), less<int>()); // 1 2 3 4 5
for_each(v.begin(), v.end(), [](int v) { cout << v << " ";});
cout << endl;
}
```

需要函数对象继承自`binary_function`。

3.4.3 成员函数适配器

```
class Person
{
public:
    string name;
    int age;
    Person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }
    void showPerson()
    {
        cout << "name = " << this->name << ", age = " << this->age << endl;
    }
};

void myPrintPerson(Person &ob)
{
    cout << "name = " << ob.name << ", age = " << ob.age << endl;
}

void test04()
{
    vector<Person> v;
    v.push_back(Person("德玛西亚", 18));
    v.push_back(Person("狗头", 28));
    v.push_back(Person("牛头", 19));
    v.push_back(Person("小法", 38));

    // 遍历 myPrintPerson普通函数
    // for_each(v.begin(), v.end(), myPrintPerson);

    // 遍历 Person成员函数
    // 利用mem_fun_ref将Person内部成员函数转成适配器
    for_each(v.begin(), v.end(), mem_fun_ref(&Person::showPerson));
}
```

3.4.4 函数指针适配器

```
void myPrint(int val, int tmp)
{
    cout << val + tmp << " ";
}
void test05()
{
    vector<int> v;
    for (int i = 0; i < 10; i++)
    {
        v.push_back(i);
    }
    // ptr_fun()把一个普通的函数指针适配成函数对象
    for_each(v.begin(), v.end(), bind2nd(ptr_fun(myPrint), 100)); // 100 101 102
103 104 105 106 107 108 109
    cout << endl;
}
```

使用函数指针适配器，就可以使用普通函数作为适配器函数。

3.5 算法概述

算法主要是有头文件组成。是所有STL头文件中最大的一个，其中常用的功能涉及到比较、交换、查找、遍历、复制、修改、反转、排序、合并等，体积很小，只包括在几个序列容器上进行的简单运算的模板函数，定义了一些模板类，用以声明函数对象。

3.6 常用遍历算法

3.6.1 for_each遍历算法

```
/*
遍历算法：遍历容器元素
@param beg 开始迭代器
@param end 结束迭代器
@param _callback 函数回调或者函数对象
@return 函数对象
*/
for_each(iterator beg, iterator end, _callback);
```

3.6.2 transform算法

```
/*
transform算法将指定容器区间元素搬运到另一容器中
注意：transform不会给目标容器分配内存，所以需要我们提前分配好内存
@param beg1 源容器开始迭代器
@param end1 源容器结束迭代器
```

```
@param beg2 目标容器开始迭代器
@param _callback 回调函数或者函数对象
@return 返回目标容器迭代器
*/
transform(iterator beg1, iterator end1, iterator beg2, _callback);
```

```
int myTransInt(int val)
{
    return val;
}

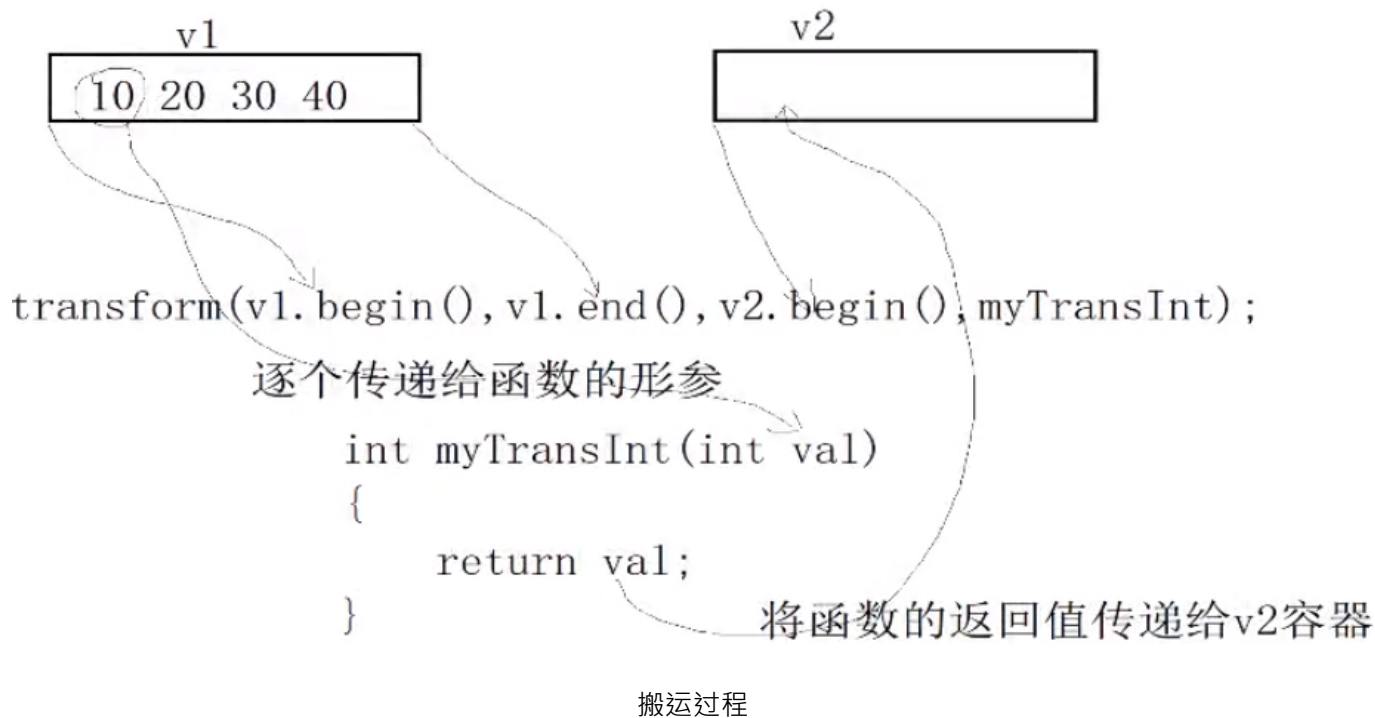
int myTransAdd(int val, int tmp)
{
    return val + tmp;
}

void test01()
{
    vector<int> v1;
    v1.push_back(10);
    v1.push_back(20);
    v1.push_back(30);
    v1.push_back(40);
    v1.push_back(50);

    // 将v1容器的元素搬运到v2中
    vector<int> v2;
    // 预先：设置v2的大小
    v2.resize(v1.size());
    transform(v1.begin(), v1.end(), v2.begin(), myTransInt);

    for_each(v2.begin(), v2.end(), [](int val){ cout << val << " ";}); // 10 20 30
    40 50
    cout << endl;

    vector<int> v3(v1.size(), 0);
    transform(v1.begin(), v1.end(), v2.begin(), bind2nd(ptr_fun(myTransAdd),
100));
    for_each(v2.begin(), v2.end(), [](int val){ cout << val << " ";}); // 110 120
130 140 150
    cout << endl;
}
```



搬运过程可以对元素进行特殊处理，比如：

```
int myTransInt(int val)
{
    return val + 1000;
}
```

3.7 常用查找算法

3.7.1 find算法

```
/*
    find算法：查找元素
    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param value 查找的元素
    return 返回查找元素的位置
*/
find(iterator beg, iterator end, value);
```

3.7.2 find_if算法

```
/*
    find_if算法 条件查找
    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param callback 回调函数或者谓词(返回bool类型的函数对象)
    @return 返回迭代器，如果未查找到则返回容器end()的迭代器
*/
```

```
*/
find_if(iterator beg, iterator end, _callback);
```

3.7.3 adjacent_find算法

```
/*
    adjacent_find算法 · 查找相邻重复元素
    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param _callback 回调函数或者谓词(返回bool类型的函数对象)
    @return 返回相邻元素的第一个位置的迭代器
*/
adjacent_find(iterator beg, iterator end, _callback);
```

```
void test03()
{
    vector<int> v1;
    v1.push_back(10);
    v1.push_back(20);
    v1.push_back(30);
    v1.push_back(30);
    v1.push_back(40);
    v1.push_back(50);
    v1.push_back(50);

    vector<int>::iterator it;
    // 对于普通数据 · 不需要回调函数
    it = adjacent_find(v1.begin(), v1.end());
    if (it != v1.end()) {
        cout << "寻找到重复的数据：" << *it << endl; // 30
    } else {
        cout << "不存在重复的数据" << endl;
    }

    // 对于自定义数据 · 需要提供operator==运算符
    vector<Person> v2;
    v2.push_back(Person("德玛西亚", 18));
    v2.push_back(Person("小法", 19));
    v2.push_back(Person("小炮", 20));
    v2.push_back(Person("小炮", 20));
    v2.push_back(Person("牛头", 21));
    vector<Person>::iterator pit;
    pit = adjacent_find(v2.begin(), v2.end());
    if (pit != v2.end()) {
        cout << "寻找到重复的数据： name = " << pit->name << ", age = " << pit->age
        << endl; // 寻找到重复的数据： name = 小炮, age = 20
    } else {
        cout << "不存在重复的数据" << endl;
    }
}
```

```
    }  
}
```

3.7.4 binary_search算法

```
/*  
    binary_search算法 · 二分查找法  
    注意：在无序序列中不可用  
    @param beg 容器开始迭代器  
    @param end 容器结束迭代器  
    @param value 查找的元素  
    @return bool 查找到返回true · 否则返回false  
*/  
bool binary_search(iterator beg, iterator end, value);
```

```
void test04()  
{  
    vector<int> v1;  
    v1.push_back(10);  
    v1.push_back(20);  
    v1.push_back(30);  
    v1.push_back(40);  
    v1.push_back(50);  
  
    bool ret = binary_search(v1.begin(), v1.end(), 30);  
    if (ret) {  
        cout << "找到了30" << endl; // 找到了30  
    } else {  
        cout << "未找到" << endl;  
    }  
}
```

3.7.5 count算法

```
/*  
    count算法 统计元素出现次数  
    @param beg 容器开始迭代器  
    @param end 容器结束迭代器  
    @param value 回调函数或者谓词(返回bool类型的函数对象)  
    @return int返回元素个数  
*/  
int count(iterator beg, iterator end, value)
```

```

void test05()
{
    vector<int> v1;
    v1.push_back(10);
    v1.push_back(20);
    v1.push_back(10);
    v1.push_back(40);
    v1.push_back(50);
    v1.push_back(10);

    int cnt = count(v1.begin(), v1.end(), 10);
    cout << "10的个数：" << cnt << endl; // 3
}

```

3.7.6 count_if算法

```

/*
    count_if算法 统计元素出现次数
    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param callback 回调函数或者谓词(返回bool类型的函数对象)
    @return int 返回元素个数
*/
count_if(iterator beg, iterator end, _callback);

```

```

void test06()
{
    vector<int> v1;
    v1.push_back(10);
    v1.push_back(20);
    v1.push_back(30);
    v1.push_back(40);
    v1.push_back(50);

    int cnt = count_if(v1.begin(), v1.end(), bind2nd(greater_equal<int>(), 20));
    cout << "大于等于20的个数：" << cnt << endl; // 4
}

```

3.8 常用排序算法

3.8.1 merge算法

```

/*
    merge算法 容器元素合并，并存储到另一容器中
    注意：两个容器必须是有序的
    @param beg1 容器1开始迭代器

```



```

@param end1 容器1结束迭代器
@param beg2 容器2开始迭代器
@param end2 容器2结束迭代器
@param dest 目标容器开始迭代器，需要预先设置目标容器的大小
*/
merge(iterator beg1, iterator end1, iterator beg2, iterator end2, iterator dest);

```

```

void test01()
{
    vector<int> v1;
    v1.push_back(1);
    v1.push_back(3);
    v1.push_back(5);
    v1.push_back(7);

    vector<int> v2;
    v2.push_back(2);
    v2.push_back(4);
    v2.push_back(6);
    v2.push_back(8);

    vector<int> v3;
    // 预先设置v3的大小
    v3.resize(v1.size() + v2.size());
    merge(v1.begin(), v1.end(), v2.begin(), v2.end(), v3.begin());
    for_each(v3.begin(), v3.end(), [](int val) { cout << val << " "; }); // 1 2 3
4 5 6 7 8
    cout << endl;
}

```

3.8.2 sort算法

```

/*
    sort算法 容器元素排序
    @param beg 容器1开始迭代器
    @param end 容器1结束迭代器
    @param _callback 回调函数或者谓词(返回bool类型的函数对象)
*/
sort(iterator beg, iterator end, _callback);

```

3.8.3 random_shuffle算法

```

/*
    random_shuffle 算法，对指定范围内的元素随机调整次序
    @param beg 容器开始迭代器
    @param end 容器结束迭代器

```

```
*/
random_shuffle(iterator beg, iterator end)
```

```
void test02()
{
    srand(time(NULL));
    vector<int> v1;
    v1.push_back(1);
    v1.push_back(3);
    v1.push_back(5);
    v1.push_back(7);

    for_each(v1.begin(), v1.end(), [](int val) { cout << val << " "; }); // 1 3 5
7
    cout << endl;

    random_shuffle(v1.begin(), v1.end());

    for_each(v1.begin(), v1.end(), [](int val) { cout << val << " "; }); // 1 7 3
5 · 每次都是1735 · 需要设置种子
    cout << endl;
}
```

3.8.4 reverse算法

```
/*
reverse算法 反转指定范围的元素
@param beg 容器开始迭代器
@param end 容器结束迭代器
*/
reverse(iterator beg, iterator end)
```

```
void test03()
{
    vector<int> v1;
    v1.push_back(1);
    v1.push_back(3);
    v1.push_back(5);
    v1.push_back(7);

    for_each(v1.begin(), v1.end(), [](int val) { cout << val << " "; }); // 1 3 5
7
    cout << endl;

    reverse(v1.begin(), v1.end());

    for_each(v1.begin(), v1.end(), [](int val) { cout << val << " "; }); // 7 5 3
```

```
1
    cout << endl;
}
```

3.9 常用拷贝和替换算法

3.9.1 copy算法

```
/*
    copy算法，将容器内指定范围的元素拷贝到另一个容器中
    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param dest 目标起始迭代器
*/
copy(iterator beg, iterator end, iterator dest)
```

```
#include <iterator>
void test04()
{
    vector<int> v1;
    v1.push_back(1);
    v1.push_back(3);
    v1.push_back(5);
    v1.push_back(7);

    for_each(v1.begin(), v1.end(), [](int val) { cout << val << " "; }); // 1 3 5
7
    cout << endl;

    vector<int> v2;
    // 预先设置大小
    v2.resize(v1.size());
    copy(v1.begin(), v1.end(), v2.begin());

    for_each(v2.begin(), v2.end(), [](int val) { cout << val << " "; }); // 1 3 5
7
    cout << endl;

    // copy秀一下：用copy输出
    copy(v2.begin(), v2.end(), ostream_iterator<int>(cout, " ")); // 1 3 5 7
}
```

可以用copy将数据输出到屏幕上，需要引入include <iterator>头文件。

3.9.2 replace算法

```

/*
    replace算法 将容器内指定范围的旧元素修改为新元素
    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param oldvalue 旧元素
    @param newvalue 新元素
*/
replace(iterator beg, iterator end, oldvalue, newvalue);

```

```

void test05()
{
    vector<int> v1;
    v1.push_back(1);
    v1.push_back(3);
    v1.push_back(5);
    v1.push_back(7);

    replace(v1.begin(), v1.end(), 3, 3000);

    copy(v1.begin(), v1.end(), ostream_iterator<int>(cout, " ")); // 1 3000 5 7
    cout << endl;
}

```

3.9.3 replace_if算法

```

/*
    replace_if算法 · 将容器内指定范围满足条件的元素替换为新元素
    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param callback 函数回调或者谓词(返回bool类型的函数对象)
    @param newvalue 新元素
*/
replace_if(iterator beg, iterator end, _callback, newvalue)

```

```

void test06()
{
    vector<int> v1;
    v1.push_back(1);
    v1.push_back(3);
    v1.push_back(5);
    v1.push_back(7);

    // 将容器中大于3替换成3000
    replace_if(v1.begin(), v1.end(), bind2nd(greater<int>(), 3), 3000);

    for_each(v1.begin(), v1.end(), [](int val) { cout << val << " "; }); // 1 3
}

```

```
3000 3000
    cout << endl;
}
```

3.9.4 swap算法

```
/*
    swap算法，互换两个容器的元素
    @param c1容器1
    @param c2容器2
*/
swap(container c1, container c2);
```

3.10 常用算术生成算法

3.10.1 accumulate算法

```
/*
    accumulate算法，计算容器元素累计总和
    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param value 累加值
*/
accumulate(iterator beg, iterator end, value);
```

```
#include <numeric>
void test07()
{
    vector<int> v1;
    v1.push_back(1);
    v1.push_back(3);
    v1.push_back(5);
    v1.push_back(7);

    int sum = accumulate(v1.begin(), v1.end(), 0);
    cout << "sum = " << sum << endl; // sum = 16
}
```

得引入`#include <numeric>`头文件。

3.10.2 fill算法

```
/*
    fill算法，向容器中添加元素
```

```

    @param beg 容器开始迭代器
    @param end 容器结束迭代器
    @param value 填充元素
*/
fill(iterator beg, iterator end, value);

```

```

void test08()
{
    vector<int> v1;
    v1.resize(10);
    // v1.reserve(10); // reserve不行，因为reserve是预留空间，但未开辟内存，此时
begin()==end()
    fill(v1.begin(), v1.end(), 100);
    copy(v1.begin(), v1.end(), ostream_iterator<int>(cout, " "));
    cout << endl;
}

```

3.11 常用集合算法

3.11.1 set_intersection算法

```

/*
    set_intersection算法，求两个set集合的交集
    注意：两个集合必须是有序序列，无序的也行
    @param beg1 容器1开始迭代器
    @param end1 容器1结束迭代器
    @param beg2 容器2开始迭代器
    @param end2 容器2结束迭代器
    @param dest 目标容器开始迭代器
    @return 目标容器的最后一个元素的迭代器地址
*/
set_intersection(iterator beg1, iterator end1, iterator beg2, iterator end2,
iterator dest)

```

```

void test01()
{
    vector<int> A;
    A.push_back(1);
    A.push_back(3);
    A.push_back(5);
    A.push_back(7);
    A.push_back(9);

    vector<int> B;
    B.push_back(7);
    B.push_back(9);
    B.push_back(2);
}

```

```

B.push_back(4);
B.push_back(6);

// 求交集
vector<int> iv; // 存放交集
iv.resize(min(A.size(), B.size()));
vector<int>::iterator it;
it = set_intersection(A.begin(), A.end(), B.begin(), B.end(), iv.begin());
// copy(iv.begin(), iv.end(), ostream_iterator<int>(cout, " ")); // 7 9 0 0 0
// 存在后面的3个0，如何解决？
// 方法一：不使用resize，而是用reserve。
copy(iv.begin(), it, ostream_iterator<int>(cout, " ")); // 7 9
cout << endl;

vector<int> uv;
uv.resize(A.size() + B.size());
it = set_union(A.begin(), A.end(), B.begin(), B.end(), uv.begin());
copy(uv.begin(), it, ostream_iterator<int>(cout, " ")); // 1 3 5 7 9 2 4 6
cout << endl;

vector<int> dv;
dv.resize(A.size());
it = set_difference(A.begin(), A.end(), B.begin(), B.end(), dv.begin());
copy(dv.begin(), it, ostream_iterator<int>(cout, " ")); // 1 3 5
cout << endl;

dv.resize(B.size());
it = set_difference(B.begin(), B.end(), A.begin(), A.end(), dv.begin());
copy(dv.begin(), it, ostream_iterator<int>(cout, " ")); // 2 4 6
cout << endl;
}

```

3.11.2 set_union算法

```

/*
    set_union算法，求两个set集合的并集
    注意：两个集合必须是有序序列，无序的也行
    @param beg1 容器1开始迭代器
    @param end1 容器1结束迭代器
    @param beg2 容器2开始迭代器
    @param end2 容器2结束迭代器
    @param dest 目标容器开始迭代器
    @return 目标容器的最后一个元素的迭代器地址
*/
set_union(iterator beg1, iterator end1, iterator beg2, iterator end2, iterator
dest)

```

3.11.3 set_difference算法

```

/*
    set_difference算法，求两个set集合的差集
    // 注意：两个集合必须是有序序列，无序的也行
    @param beg1 容器1开始迭代器
    @param end1 容器1结束迭代器
    @param beg2 容器2开始迭代器
    @param end2 容器2结束迭代器
    @param dest 目标容器开始迭代器
    @return 目标容器的最后一个元素的迭代器地址
*/
set_difference(iterator beg1, iterator end1, iterator beg2, iterator end2,
iterator dest)

```

3.12 STL综合案例

演讲比赛：

```

#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <unistd.h>
#include <deque>
#include <cstring>
#include <numeric>
#include <map>
using namespace std;

class Speaker
{
public:
    string name;
    int score[3];
public:
    Speaker(){}
    Speaker(string name)
    {
        this->name = name;
        memset(score, 0, sizeof(score));
    }
};

void createSpeaker(vector<int> &v, map<int, Speaker> &m)
{
    for (int i = 0; i < 24; i++)
    {
        // 存放选手编号
        v.push_back(100 + i);
        string name = "选手";
        name += 'A' + i;
    }
}

```



```

        m.insert(make_pair(i+100, Speaker(name)));
    }
}

void speech_contest(int epoch, vector<int> &v, map<int, Speaker> &m, vector<int>
&v1)
{
    int count = 0;
    // 设计一个<分数·编号>容器
    multimap<int, int, greater<int>> m2;
    // 选手逐一登台比赛
    for (vector<int>::iterator it = v.begin(); it != v.end(); it++) {
        count++;
        deque<int> d;
        for (int i = 0; i < 10; i++) { // 10个评委打分
            int score = rand() % 41 + 60;
            d.push_back(score);
        }
        // 对d容器排序
        sort(d.begin(), d.end());
        // 去掉最高分·去掉最低分
        d.pop_back();
        d.pop_front();
        // 求取总分数
        int sum = accumulate(d.begin(), d.end(), 0);

        int avg = sum / d.size();
        m[*it].score[epoch - 1] = avg;
        m2.insert(make_pair(avg, *it));

        if (count % 6 == 0) { // 刚好6人·把上面的6人的成绩取3
            // 90 80 70 60 50 40
            int i = 0;
            for (multimap<int, int, greater<int>>::iterator mit = m2.begin(); mit
!= m2.end() && i < 3; mit++, i++) {
                v1.push_back(mit->second);
            }
            m2.clear(); // 清空
        }
    }
}

/*
18690705637
*/

void printSpeechResult(int epoch, vector<int> &v, map<int, Speaker> &m,
vector<int> &v1)
{
    cout << "第" << epoch << "轮比赛成绩如下" << endl;
    int count = 0;
    int grp = 0;
    for (vector<int>::iterator it = v.begin(); it != v.end(); it++) {
        if (count % 6 == 0) {

```

```

        grp = count/6 + 1;
        cout << "第" << grp << "组的成绩如下：" << endl;
    }

    cout << "\t姓名：" << m[*it].name << "，得分：" << m[*it].score[epoch - 1]
<< endl;

    count++;

    // 每个组的成绩打印完，立马打印晋级名单
    if (count % 6 == 0) {
        cout << "第" << grp << "组晋级名单如下：" << endl;
        int cnt = 0;
        for (vector<int>::iterator vit = v1.begin() + (grp - 1)*3; vit !=
v1.end() && cnt < 3; vit++, cnt++) {
            cout << "\t姓名：" << m[*vit].name << "，得分：" <<
m[*vit].score[epoch-1] << endl;
        }
    }
}

int main(int argc, char **argv)
{
    // 创建24名选手，将选手<编号，选手>放入map容器中，选手编号放vector<>容器中
    vector<int> v; // 选手编号
    map<int, Speaker> m; // <编号，选手>
    createSpeaker(v, m);

    // for_each(m.begin(), m.end(), [](const pair<int, Speaker> &p){
    //     cout << p.first << "，" << p.second.name << endl;
    // });

    // 设置种子
    srand(time(NULL));
    // 第一轮：参加的选手抽签
    random_shuffle(v.begin(), v.end());
    // 进行第一轮比赛
    // 1表示当前轮数，v选手编号，m选手信息，v1晋级容器
    vector<int> v1;
    speech_contest(1, v, m, v1);
    // 打印第一轮比赛结果：所有参与比赛的成绩 晋级的名单
    printSpeechResult(1, v, m, v1);

    cout << "=====" << endl;
    // 第二轮：参加的选手抽签
    random_shuffle(v1.begin(), v1.end());
    // 进行第二轮比赛
    // 2表示当前轮数，v选手编号，m选手信息，v1晋级容器
    vector<int> v2;
    speech_contest(2, v1, m, v2);
    // 打印第二轮比赛结果：所有参与比赛的成绩 晋级的名单
    printSpeechResult(2, v1, m, v2);
}

```

```
    cout << "===== " << endl;
    // 第三轮：参加的选手抽签
    random_shuffle(v2.begin(), v2.end());
    // 进行第三轮比赛
    // 3表示当前轮数，v选手编号，m选手信息，v1晋级容器
    vector<int> v3;
    speech_contest(3, v2, m, v3);
    // 打印第一轮比赛结果：所有参与比赛的成绩 晋级的名单
    printSpeechResult(3, v2, m, v3);
    return 0;
}
```